

The objectives of cursors and triggers in PL/SQL

The objectives of *cursors* and *triggers* in PL/SQL are distinct, as each serves a unique role in database management:



Objective of a Cursor



Facilitate Row-Based Data Handling: Cursors allow developers to handle large result sets row-by-row, which is especially useful for performing specific actions on each record.

Reduce Database Load: By retrieving rows incrementally rather than all at once, cursors help reduce memory usage and manage resources more efficiently.

Improve Transaction Control: Cursors enable fine-grained transaction management, allowing applications to apply conditional logic or validations on individual rows during a single operation.

Objective of a Trigger



Automate Database Tasks: Triggers execute automatically in response to specific database events, reducing the need for manual intervention and application code.

Ensure Data Accuracy and Integrity: Triggers enforce critical rules and constraints, such as validating data before insertions or updates, ensuring that only valid data is entered into the database.

Support Auditing and Security: Triggers can log changes, record audit trails, and enforce security measures by validating data against predefined conditions.

Simplified Perspective



Cursors focus on controlled and step-by-step data retrieval.

Triggers focus on automating processes and enforcing rules to maintain data integrity when data changes.

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | ericmaniraguha2024@gmail.com | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

November 2024

Database development with PL/SQL

Reference reading



- [Difference between Cursor and Trigger in DBMS](#)
- [Difference between Cursor and Trigger in PLSQL](#)
- [9 Using Triggers](#)
- [CREATE TRIGGER](#)
- [Oracle PL/SQL Trigger: A Comprehensive Guide](#)
- [Auditing employee table changes with PL/SQL Trigger](#)

Lecture 09 – Cursor and Trigger - Audit

Introduction to Cursors

1. In Oracle PL/SQL, a **cursor** is a pointer to the result set of a SQL query. Cursors allow developers to process individual rows returned by a query sequentially.
2. Cursor is a pointer to a memory area called context area (A memory region inside the Process Global Area or PGA) assigned to hold information about the processing of SELECT statement or DML Statement.

Here's a breakdown of cursor types in Oracle PL/SQL, along with syntax and examples:

Context Area

Is a memory region inside the PGA which helps oracle server in processing an SQL statement by holding the important information about that statement.

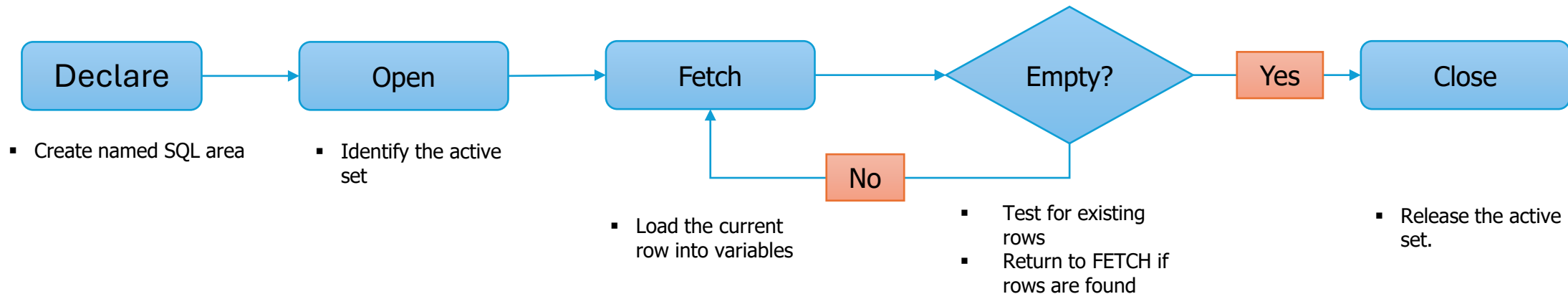
Using cursor you can control the context area as it is a pointer to the same.

Types of Cursors

Implicit Cursor: Automatically created by Oracle for **single-row queries** and **DML operations** (like INSERT, UPDATE, or DELETE). Implicit cursors are managed by Oracle, and their details are accessible only through cursor attributes (like %ROWCOUNT, %FOUND).

Explicit Cursor: Defined by the programmer, allowing more control over multi-row queries. These are created in the DECLARE section of a PL/SQL block.

Cursor Implementation with PL/SQL



Declare: Start by declaring the cursor, specifying the SQL query that the cursor will use to retrieve data. This step sets up the cursor so it can be used in the rest of the process.

Open: Open the cursor to establish a connection to the database and prepare it for fetching rows. This initializes the cursor's position before data retrieval. When you open a cursor the memory will be allocated to it.

Fetch: Retrieve the next row from the cursor into a variable or record. This step will continue as long as there are rows remaining in the result set.

Empty: After each fetch, check if the cursor is empty (no more rows to fetch). If there are more rows, go back to the **Fetch** step. If the cursor is empty, proceed to the next step.

Close: Close the cursor once all rows are fetched, releasing any resources associated with it. This step marks the end of the cursor's lifecycle.

Application of Cursors

Applications of Cursors

Cursors in PL/SQL are used to process multiple rows returned by SQL queries one row at a time, which is useful when you need to manipulate data row-by-row. Cursors can be either **explicit** (declared and controlled by the programmer) or **implicit** (automatically created by Oracle for single-row queries).

Common Applications of Cursors

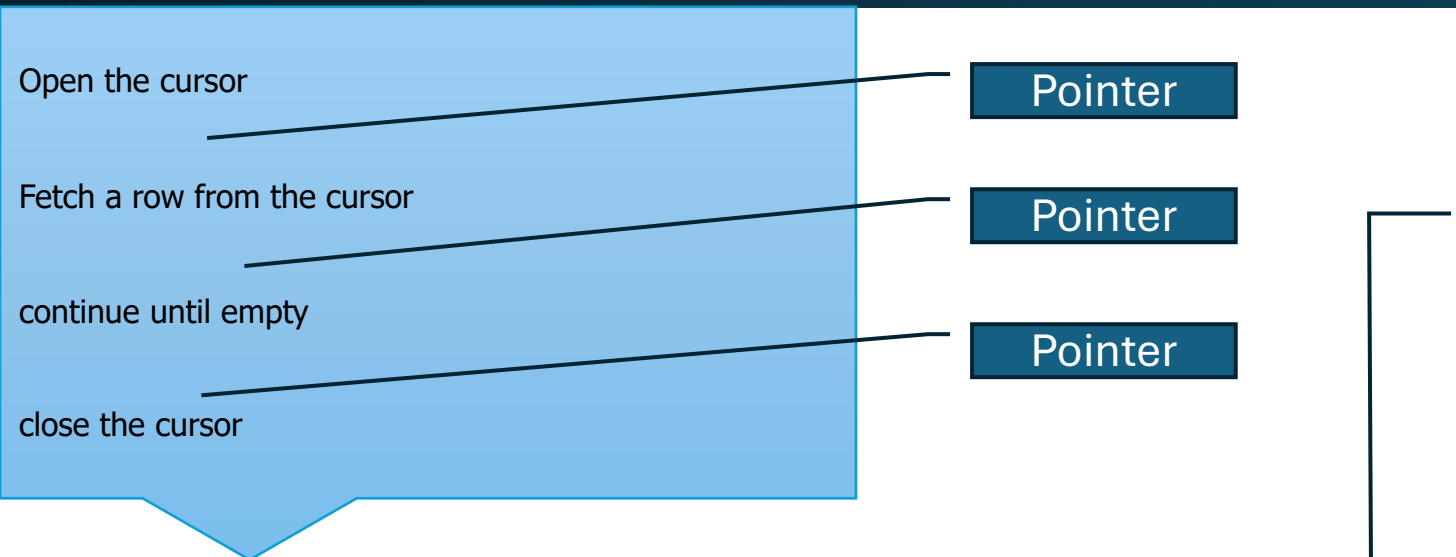
Row-by-Row Processing: Useful when you need to perform **calculations or data manipulation for each row individually**, such as when iterating through orders to apply specific business logic for each one.

Generating Reports: Loop through each record in a result set to gather data or perform complex aggregation calculations, often for generating detailed reports.

Bulk Data Transfer or Processing: Move data from one table to another or transform data before insertion by fetching rows, processing each row, and inserting into a new table.

Complex Calculations: Perform detailed calculations that involve data from multiple rows and potentially multiple tables.

Introduction to Cursor Syntax



Explicit Cursor (Basic Programming Structure)

- **Open the Cursor:** Initializes the cursor and sets it to the first row in the result set.
- **Fetch a Row:** Retrieves the current row and advances the cursor to the next row.
- **Continue Until Empty:** Repeat fetching until no more rows are left.
- **Close the Cursor:** Releases the cursor, freeing up resources.

```
DECLARE
-- Step 1: Declare the cursor with a SELECT statement
CURSOR cursor_name IS
  SELECT column1, column2, ...
  FROM table_name
  WHERE condition;

-- Variables to hold the data from each row
var1 table_name.column1%TYPE;
var2 table_name.column2%TYPE;
-- Add more variables as needed

BEGIN
-- Step 2: Open the cursor
OPEN cursor_name;

-- Step 3: Fetch rows from the cursor
LOOP
  FETCH cursor_name INTO var1, var2;

  -- Exit the loop when no more rows are found
  EXIT WHEN cursor_name%NOTFOUND;

  -- Process each row (e.g., display values or perform calculations)
  DBMS_OUTPUT.PUT_LINE('Column1: ' || var1 || ', Column2: ' || var2);
END LOOP;

-- Step 4: Close the cursor
CLOSE cursor_name;
END;
/
```

Implicit cursor example and its purpose

Example	SQL Statement	Explanation
Update Statement	UPDATE Employees SET Salary = Salary * 1.1 WHERE Department = 'IT';	Increases the salary of all employees in the IT department by 10%. An implicit cursor is created automatically, which finds and updates these rows based on the WHERE clause, then closes automatically.
Select Into Statement	DECLARE total_employees NUMBER; BEGIN SELECT COUNT(*) INTO total_employees FROM Employees; END; /	Retrieves the total count of rows in the Employees table and stores it in the total_employees variable. An implicit cursor is used to handle the retrieval operation and closes right after execution.
Insert Statement	INSERT INTO Orders (OrderID, CustomerID, OrderDate) VALUES (123, 'C001', SYSDATE);	Inserts a new row into the Orders table with values for OrderID, CustomerID, and OrderDate. The implicit cursor processes this single-row insertion and closes automatically after the operation.
Delete Statement	DELETE FROM Customers WHERE Country = 'USA';	Deletes all rows in the Customers table where the Country is 'USA'. The implicit cursor locates these rows, performs the deletion, and then closes automatically.

Explanation of Implicit Cursors in Oracle PL/SQL

- **Implicit Cursors:** In Oracle PL/SQL, implicit cursors are created automatically by the database whenever a DML (Data Manipulation Language) operation like UPDATE, INSERT, DELETE, or a SELECT INTO query is executed.
- **Automatic Management:** Oracle opens, processes, and closes these cursors internally without user intervention, which simplifies handling of single-row or straightforward multi-row operations.
- **Efficiency:** Implicit cursors are generally more efficient than explicit cursors because they do not require explicit declaration, opening, fetching, or closing by the user.

Description of each cursor attributes

In Oracle PL/SQL, **cursor attributes** provide essential information about the behavior and status of cursors during operations. These attributes can be used with both implicit and explicit cursors, and they help to handle and monitor cursor operations effectively. Here's a detailed description of each cursor attribute:

Cursor Attribute	Description
%FOUND	Returns TRUE if the most recent FETCH operation successfully retrieved a row. If no row was fetched, it returns FALSE. For implicit cursors, %FOUND will return TRUE if a DML operation affected at least one row.
%NOTFOUND	Returns TRUE if the most recent FETCH operation did not retrieve a row (i.e., the end of the cursor's result set is reached). If a row is fetched successfully, it returns FALSE. This is the opposite of %FOUND.
%ISOPEN	Returns TRUE if the cursor is open. For implicit cursors, this attribute always returns FALSE because Oracle automatically opens and closes implicit cursors. Explicit cursors, however, require manual control, so %ISOPEN can be useful to check whether an explicit cursor is open before performing further operations.
%ROWCOUNT	Returns the number of rows affected by a DML operation (for implicit cursors) or the number of rows fetched so far (for explicit cursors). %ROWCOUNT gives the cumulative count of rows processed by the cursor since it was opened.

Implicit Cursor:

1. SQL%OPEN
2. SQL%FOUND
3. SQL%NOTFOUND

1. SQL%ISOPEN

- **Explanation:** SQL%ISOPEN checks if an implicit cursor is open. For implicit cursors, it always returns FALSE since the cursor automatically closes once the SQL statement executes.
- **Example:** After executing a DELETE on the ALLOWANCES table, SQL%ISOPEN will return FALSE immediately, as the cursor closes.

2. SQL%FOUND and SQL%NOTFOUND

- **Explanation:** These attributes are useful for determining whether an operation affected any rows.



```
CREATE OR REPLACE PROCEDURE delete_allowance(p_allowance_id NUMBER) AS
BEGIN
    -- Attempt to delete a row from the ALLOWANCES table with a specific
    ALLOWANCE_ID.
    DELETE FROM ALLOWANCES WHERE ALLOWANCE_ID = p_allowance_id;

    -- Check if the DELETE statement affected any rows.
    IF SQL%FOUND THEN
        -- If a row was deleted, output a success message with the allowance ID.
        DBMS_OUTPUT.PUT_LINE('Allowance with ID ' || p_allowance_id || ' deleted
        successfully.');
```

Use Case: If p_allowance_id matches a record in ALLOWANCES, SQL%FOUND is TRUE, and it outputs success; if not, SQL%NOTFOUND is implicitly TRUE, and it outputs "No allowance found."

Implicit Cursor:

1. SQL%FOUND, 2. SQL%ROWCOUNT

```
-- Call the delete_allowance procedure with a specific  
ALLOWANCE_ID, for example 101.  
BEGIN  
    delete_allowance(101);  
END;  
/
```

**Combining
SQL%FOUND,
SQL%NOTFOUND,
and
SQL%ROWCOUNT**

SQL%FOUND: Returns TRUE if the last DML operation affected one or more rows.

SQL%NOTFOUND: Returns TRUE if the last DML operation did not affect any rows (opposite of SQL%FOUND).

SQL%ROWCOUNT: Provides the exact number of rows affected by the last DML operation.

```
CREATE OR REPLACE PROCEDURE delete_allowance(p_allowance_id NUMBER) AS  
BEGIN  
    -- Attempt to delete a row from the ALLOWANCES table with a specific ALLOWANCE_ID.  
    DELETE FROM ALLOWANCES WHERE ALLOWANCE_ID = p_allowance_id;  
  
    -- Use SQL attributes to check the outcome of the DELETE operation  
    IF SQL%FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('Allowance with ID ' || p_allowance_id || ' deleted successfully.');        DBMS_OUTPUT.PUT_LINE('Rows deleted: ' || SQL%ROWCOUNT);  
    ELSE  
        DBMS_OUTPUT.PUT_LINE('No allowance found with ID ' || p_allowance_id);  
    END IF;  
  
    EXCEPTION  
        WHEN NO_DATA_FOUND THEN  
            -- Handle case where SELECT INTO returns no rows  
            DBMS_OUTPUT.PUT_LINE('No data found for the given ID.');        WHEN TOO_MANY_ROWS THEN  
            -- Handle case where SELECT INTO returns multiple rows  
            DBMS_OUTPUT.PUT_LINE('Multiple rows found for the given ID.');        WHEN OTHERS THEN  
            -- Handle all other exceptions  
            DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);  
    END;  
/
```

Using a Cursor to Fetch
Employees by Department

Implicit Cursor:

1. SQL%NOTFOUND,
2. SQL%ROWCOUNT

```
-- Enable output
SET SERVEROUTPUT ON SIZE 100000;
```

```
DECLARE
```

```
  v_employee_id employees.employee_id%TYPE; -- Employee ID
  v_first_name employees.first_name%TYPE;
  v_last_name employees.last_name%TYPE;
```

```
BEGIN
```

```
  -- Attempt to select an employee from department 30
```

```
  SELECT employee_id, first_name, last_name
  INTO v_employee_id, v_first_name, v_last_name
  FROM employees
  WHERE department_id = 30 AND ROWNUM = 1;
```

```
  -- Check if no employee was found
```

```
  IF SQL%NOTFOUND THEN
```

```
    DBMS_OUTPUT.PUT_LINE('No employee found in department 30.');
```

```
    RETURN; -- Exit the block if no employee is found
```

```
  END IF;
```

```
  -- Attempt to delete the selected employee
```

```
  DELETE FROM employees WHERE employee_id = v_employee_id;
```

```
  -- Check if the delete operation didn't find or affect any rows
```

```
  IF SQL%NOTFOUND THEN
```

```
    DBMS_OUTPUT.PUT_LINE('No employee with ID ' || v_employee_id || ' was found for
deletion.');
```

```
  ELSE
```

```
    DBMS_OUTPUT.PUT_LINE('Employee ID ' || v_employee_id || ' deleted successfully.');
```

```
  END IF;
```

```
  -- Output the number of rows deleted
```

```
  DBMS_OUTPUT.PUT_LINE('Rows deleted: ' || SQL%ROWCOUNT);
```

```
EXCEPTION
```

```
  -- Handle unexpected errors
```

```
  WHEN OTHERS THEN
```

```
    DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
```

```
END;
```

```
/
```

Question - Implicit Cursor SQL%NOTFOUND



Write a PL/SQL block to retrieve the allowance details of an employee from a specific department and handle potential errors using implicit cursors.

Requirements:

- Retrieve the ALLOWANCE_ID, ALLOWANCE_AMOUNT, and EFFECTIVE_DATE for an employee in **Department 30**.
- The employee should be identified by the ROLE_ID in the EMPLOYEES table.
- Use an implicit cursor to perform the query and retrieve the data.
- Display the details of the allowance (allowance ID, amount, and effective date) using DBMS_OUTPUT.PUT_LINE.
- After retrieving the data, use the SQL%NOTFOUND attribute to check if the query did not return any results:
 - If no allowance is found for the employee in Department 30, output the message: **"No allowance found for employee in department 30."**
 - If an allowance is found, output the message: **"Allowance found for employee in department 30."**

Exception Handling:

- If no allowance is found (using SQL%NOTFOUND), display: **"No allowance found for employee in department 30."**
- If more than one allowance record is returned unexpectedly (using SQL%ROWCOUNT or SQLERRM), display a custom message: **"Too many allowances found for employee in department 30."**
- For any other error, display the error message using SQLERRM.

Expected behavior:

- If no allowance is found for an employee in Department 30, the block will output: "No allowance found for employee in department 30."
- If multiple allowances are found, the block will output: "Too many allowances found for employee in department 30."
- If one allowance is found, the block will display the details of the allowance along with the message: "Allowance found for employee in department 30."



Solution –

Implicit Cursor

1. SQL%NOTFOUND
2. SQL%ROWCOUNT

Using a Cursor to
Fetch Employees by
Department

```
SET SERVEROUTPUT ON SIZE 100000;

DECLARE
    -- Variables to store allowance details
    v_allowance_id ALLOWANCES.ALLOWANCE_ID%TYPE;
    v_allowance_amount ALLOWANCES.ALLOWANCE_AMOUNT%TYPE;
    v_effective_date ALLOWANCES.EFFECTIVE_DATE%TYPE;
    v_employee_role_id EMPLOYEES.ROLE_ID%TYPE;

BEGIN
    -- Retrieve the ROLE_ID for an employee in Department 30
    SELECT role_id
    INTO v_employee_role_id
    FROM employees
    WHERE department_id = 30 AND ROWNUM = 1;

    -- Attempt to select allowance details for the employee from the ALLOWANCES table
    SELECT allowance_id, allowance_amount, effective_date
    INTO v_allowance_id, v_allowance_amount, v_effective_date
    FROM allowances
    WHERE role_id = v_employee_role_id
    AND SQL%ROWCOUNT = 1; -- Ensure only one record is returned

    -- Check if the SELECT query returned no data
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('No allowance found for employee in department 30.');
```


PL/SQL Function with %ROWCOUNT and %NOTFOUND

```
CREATE OR REPLACE PROCEDURE get_salary_statistics(
    p_department_id IN NUMBER,
    p_start_date IN DATE,
    p_end_date IN DATE,
    p_salary_threshold IN NUMBER
)
IS
    v_total_salary NUMBER;
    v_average_salary NUMBER;
    v_employee_count NUMBER;
    v_salary_message VARCHAR2(200);
BEGIN
    -- Calculate the total salary of employees in the department hired between p_start_date and p_end_date
    SELECT NVL(SUM(salary), 0)
    INTO v_total_salary
    FROM employees
    WHERE department_id = p_department_id
    AND hire_date BETWEEN p_start_date AND p_end_date;

    -- Check if no rows were returned for the total salary calculation
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employees found in department ' || p_department_id || ' for the given date range.');
```

```
-- Exit if no employees are found
        RETURN;
    END IF;

    -- Calculate the average salary of employees in the department hired between p_start_date and p_end_date
    SELECT NVL(AVG(salary), 0)
    INTO v_average_salary
    FROM employees
    WHERE department_id = p_department_id
    AND hire_date BETWEEN p_start_date AND p_end_date;

    -- Check if no rows were returned for the average salary calculation
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employees found in department ' || p_department_id || ' for the given date range.');
```

```
-- Exit if no employees are found
        RETURN;
    END IF;

    -- Count employees who earn above the salary threshold
    SELECT COUNT(*)
    INTO v_employee_count
    FROM employees
    WHERE department_id = p_department_id
    AND hire_date BETWEEN p_start_date AND p_end_date
    AND salary > p_salary_threshold;

    -- Prepare the result message
    v_salary_message := 'Department ' || p_department_id || ': ' ||
        'Total Salary = ' || v_total_salary || ', ' ||
        'Average Salary = ' || v_average_salary || ', ' ||
        'Employees above salary threshold (' || p_salary_threshold || '): ' ||
        v_employee_count ||
        ', Rows processed: ' || SQL%ROWCOUNT;

    -- Output the message
    DBMS_OUTPUT.PUT_LINE(v_salary_message);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employees found for the given criteria in department ' || p_department_id);
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
```

Implicit Cursor SQL%ROWCOUNT

```
--Call the check_attendance procedure
BEGIN
    check_attendance(p_employee_id => 101, p_date =>
TO_DATE('2024-11-01', 'YYYY-MM-DD'));
END;
/
```

```
-- Enable output
SET SERVEROUTPUT ON SIZE 100000;
```

```
CREATE OR REPLACE PROCEDURE check_attendance(p_employee_id NUMBER, p_date DATE) AS
    v_attendance_status ATTENDANCE.STATUS%TYPE; -- Variable to hold the attendance status
BEGIN
    -- Implicit cursor to select the attendance status of an employee on a specific date
    SELECT STATUS
    INTO v_attendance_status
    FROM ATTENDANCE
    WHERE EMPLOYEE_ID = p_employee_id AND ATTENDANCE_DATE = p_date;

    -- Check the number of rows returned by the implicit cursor
    IF SQL%ROWCOUNT = 1 THEN
        -- Output the attendance status if a record is found
        DBMS_OUTPUT.PUT_LINE('Attendance status for Employee ID ' || p_employee_id ||
            ' on ' || TO_CHAR(p_date, 'DD-MON-YYYY') || ': ' || v_attendance_status);
    ELSE
        DBMS_OUTPUT.PUT_LINE('No attendance record found for Employee ID ' || p_employee_id ||
            ' on ' || TO_CHAR(p_date, 'DD-MON-YYYY') || '.');
    END IF;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            -- This exception will be triggered if no record is found for the specified employee and date
            DBMS_OUTPUT.PUT_LINE('No attendance record found for Employee ID ' || p_employee_id ||
                ' on ' || TO_CHAR(p_date, 'DD-MON-YYYY') || '.');
        WHEN OTHERS THEN
            -- Handle any other exceptions
            DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
    END;
/
```

- **Purpose:** Checks an employee's attendance on a specified date.
 - **Logic:** Retrieves **STATUS** from **ATTENDANCE** where **EMPLOYEE_ID** and **ATTENDANCE_DATE** match inputs.
 - **Output:** Displays attendance status if found, else a message stating no record exists.
- Exceptions:**
- **NO_DATA_FOUND:** No matching attendance record.
 - **OTHERS:** Handles any other errors.

Implicit Cursor SQL%BULK_ROWCOUNT

```
-- Returned by %BULK_ROWCOUNT  
Deletion in Department #10: 5 employees deleted.  
Deletion in Department #20: 3 employees deleted.  
Deletion in Department #30: 2 employees deleted.
```

```
-- Returned by %ROWCOUNT  
Total employees deleted: 10
```

- **Explanation:** Useful in FORALL statements to determine the number of rows affected by each execution in bulk operations.
- **Example:** Deleting employee records based on department, with feedback on each department's deletions.

```
DECLARE  
  -- Define a nested table type to store a list of department IDs  
  TYPE DeptList IS TABLE OF NUMBER;  
  
  -- Initialize a nested table with sample Department IDs  
  depts DeptList := DeptList(10, 20, 30); -- Sample Department IDs  
BEGIN  
  -- Use FORALL to perform a bulk delete operation for each department ID in the depts table  
  FORALL i IN depts.FIRST .. depts.LAST  
    DELETE FROM EMPLOYEES WHERE DEPARTMENT_ID = depts(i);  
  -- Loop through each department ID to display the number of rows deleted for each  
  department  
  FOR i IN depts.FIRST .. depts.LAST LOOP  
    DBMS_OUTPUT.PUT_LINE('Deletion in Department #' || depts(i) || ': ' ||  
SQL%BULK_ROWCOUNT(i) || ' employees deleted.');
```

Use Case: SQL%BULK_ROWCOUNT provides the **row count for each delete operation** per department, and **SQL%ROWCOUNT** gives the total number of rows deleted.

Implicit Cursor SQL%BULK_ROWCOUNT

```
DECLARE
  -- Define a PL/SQL collection type to hold employee IDs
  TYPE emp_id_array IS TABLE OF employees.employee_id%TYPE;

  -- Initialize an array of employee IDs to be used in the bulk operation
  emp_ids emp_id_array := emp_id_array(100, 101, 102); -- Employee IDs for updating
BEGIN
  -- Using FORALL to perform a bulk update on multiple employees' salaries
  -- This loop increases the salary by 10% for each employee in emp_ids
  FORALL i IN emp_ids.FIRST .. emp_ids.LAST
    UPDATE employees
      SET salary = salary * 1.1 -- Applying a 10% salary raise
      WHERE employee_id = emp_ids(i);

  -- Loop through the collection to output the number of rows affected for each employee ID
  FOR i IN emp_ids.FIRST .. emp_ids.LAST LOOP
    -- Display the number of rows affected for each employee update using
    SQL%BULK_ROWCOUNT
      DBMS_OUTPUT.PUT_LINE('Rows affected for employee_id ' || emp_ids(i) || ': ' ||
    SQL%BULK_ROWCOUNT(i));
  END LOOP;
END;
/
```

Key Points

- SQL%BULK_ROWCOUNT is only available in FORALL statements.
- It helps to measure individual execution results within a bulk DML operation.
- It is particularly useful for **error logging** and **auditing row-level results** in bulk processing.

Using SQL%BULK_ROWCOUNT efficiently can lead to better performance and more granular control when handling bulk data modifications.

Implicit Cursor SQL%BULK_ROWCOUNT

```
DECLARE
-- Define a record type for department details
TYPE DeptRecord IS RECORD (
    DEPARTMENT_NAME VARCHAR2(100),
    LOCATION         VARCHAR2(100),
    COUNTRY_ID        NUMBER
);
-- Define a nested table type of the DeptRecord type
TYPE DeptList IS TABLE OF DeptRecord;

-- Initialize the nested table with department details
depts DeptList := DeptList(
    DeptRecord('Marketing', 'New York', 1),
    DeptRecord('Finance', 'London', 2),
    DeptRecord('HR', 'Sydney', 3)
);
BEGIN
-- Perform bulk insert on the DEPARTMENTS table for each department in the depts table
FORALL i IN depts.FIRST .. depts.LAST
    INSERT INTO DEPARTMENTS (DEPARTMENT_NAME, LOCATION, COUNTRY_ID)
    VALUES (depts(i).DEPARTMENT_NAME, depts(i).LOCATION, depts(i).COUNTRY_ID);

-- Loop through each department to output the insertion status for each record
FOR i IN depts.FIRST .. depts.LAST LOOP
    DBMS_OUTPUT.PUT_LINE('Insertion for Department: ' || depts(i).DEPARTMENT_NAME ||
        ', Location: ' || depts(i).LOCATION ||
        ', Country ID: ' || depts(i).COUNTRY_ID);
END LOOP;

-- Display the total number of departments inserted
DBMS_OUTPUT.PUT_LINE('Total departments inserted: ' || SQL%BULK_ROWCOUNT);
END;
/
```

Key Changes:

- **DeptRecord**: A RECORD type is defined for each department's details (name, location, and country ID).
- **DeptList**: A TABLE type of DeptRecord is defined to store multiple departments.
- **Initializing depts**: The nested table depts is initialized with DeptRecord values for each department, ensuring that each department's details are inserted correctly.

Explanation:

- **Record Type**: A RECORD type holds a single department's data (name, location, country ID).
- **Table Type**: A TABLE type (DeptList) is used to hold multiple RECORD entries.
- **FORALL Statement**: This is used for efficient bulk insert into the DEPARTMENTS table.
- **DBMS_OUTPUT.PUT_LINE**: This outputs the status of each department insertion.
- **SQL%BULK_ROWCOUNT**: This displays the total number of rows inserted at the end.

This code should now compile and run without errors, assuming that the DEPARTMENTS table already exists with the appropriate columns (DEPARTMENT_NAME, LOCATION, COUNTRY_ID) and DEPARTMENT_ID is auto-generated.

Summary:

- The bulk operation in this block is the use of **FORALL**, which efficiently inserts multiple rows from the depts nested table into the DEPARTMENTS table in a single operation. This method reduces context switching and improves performance for large insertions.
- You are also using **SQL%BULK_ROWCOUNT** after the bulk operation to display how many rows were inserted, confirming the success of the bulk operation.

Implicit Cursor SQL%BULK_EXCEPTIONS

```
DECLARE
-- Define a collection type to hold a list of employee IDs as numbers
TYPE employee_ids_t IS TABLE OF NUMBER;
-- Initialize the collection with sample employee IDs
employee_ids employee_ids_t := employee_ids_t(101, 102, 103, 104, 105);

-- Variable to store the count of errors that occurred during the bulk operation
errors NUMBER;
-- Variable to use in loops for error handling
i NUMBER;
BEGIN
-- Perform bulk INSERT operation on the 'employees' table
-- Using 'SAVE EXCEPTIONS' to capture individual row errors and continue with other rows
FORALL j IN employee_ids.FIRST .. employee_ids.LAST SAVE EXCEPTIONS
    INSERT INTO employees (employee_id, salary)
    VALUES (employee_ids(j), 5000); -- Assign a salary of 5000 for each employee ID
-- Exception handling block to catch and process any errors
EXCEPTION
WHEN OTHERS THEN
    -- Get the count of exceptions that occurred during the FORALL operation
    errors := SQL%BULK_EXCEPTIONS.COUNT;

    -- Output the number of errors encountered
    DBMS_OUTPUT.PUT_LINE('Number of errors: ' || errors);

    -- Loop through each exception recorded in SQL%BULK_EXCEPTIONS
    FOR i IN 1 .. errors LOOP
        -- Output the index of the employee ID that caused the error and the error code
        DBMS_OUTPUT.PUT_LINE('Error at index ' ||
            SQL%BULK_EXCEPTIONS(i).ERROR_INDEX ||
            ': Error code ' ||
            SQL%BULK_EXCEPTIONS(i).ERROR_CODE);
    END LOOP;
END;
```

Explanation of Key Sections

- **DECLARE Section:** This section defines the `employee_ids_t` type, initializes a collection `employee_ids`, and declares variables for error handling.
- **FORALL Statement:** The `FORALL` statement attempts to insert each employee ID from the collection into the `employees` table with a salary of 5000. The `SAVE EXCEPTIONS` clause is used to allow the operation to continue even if errors occur on individual rows.
- **Exception Handling:** The `EXCEPTION` block captures errors that arise from the `FORALL` operation. The `SQL%BULK_EXCEPTIONS` cursor attribute provides details about each error, including the error index (the row that caused the error) and the error code.
- **Output Error Information:** The loop within the `EXCEPTION` block outputs each error's index and code, allowing the developer to diagnose which specific rows failed and why.

This code is useful in scenarios where you want bulk processing to continue even if some records cause errors, and you need detailed error information for each failed row.

What Happens Internally:

1. The `FORALL` statement tries to insert rows:
 - `INSERT INTO employees (101, 5000)` → **Error (primary key violation).**
 - `INSERT INTO employees (102, 5000)` → **Success.**
 - `INSERT INTO employees (103, 5000)` → **Error (primary key violation).**
 - `INSERT INTO employees (104, 5000)` → **Success.**
 - `INSERT INTO employees (105, 5000)` → **Success.**
2. Errors are saved, and successful inserts are committed.
3. The exception-handling block processes and outputs the errors, iterating over `SQL%BULK_EXCEPTIONS`.

Number of errors: 2
Error at index 1: Error code 1
Error at index 3: Error code 1

Implicit Cursor SQL%BULK_EXCEPTIONS

- **Explanation:** This attribute stores errors encountered during a **FORALL** operation when using the **SAVE EXCEPTIONS** clause.
- **Example:** This example will handle errors gracefully, allowing successful records to be inserted and providing details on any that failed.

```
DECLARE
-- Define a record type for employee details
TYPE EmpRecord IS RECORD (
    FIRST_NAME VARCHAR2(50),
    LAST_NAME  VARCHAR2(50),
    EMAIL      VARCHAR2(100),
    PHONE_NUMBER VARCHAR2(15),
    HIRE_DATE   DATE,
    DEPARTMENT_ID NUMBER,
    ROLE_ID     NUMBER,
    SALARY      NUMBER,
    BONUS       NUMBER
);

-- Define a nested table type of the EmpRecord type
TYPE EmpList IS TABLE OF EmpRecord;

-- Initialize the nested table with employee details
employees EmpList := EmpList(
    EmpRecord('John', 'Doe', 'johndoe@example.com', '123-456-7890', SYSDATE, 10, 1, 50000, 5000),
    EmpRecord('Jane', 'Smith', 'janesmith@example.com', '098-765-4321', SYSDATE, 20, 2, 60000, 6000),
    EmpRecord('Robert', 'Brown', 'robertbrown@example.com', '555-555-5555', SYSDATE, 30, 3, 70000,
7000)
-- Add more records as needed
);

-- Variable to count errors
errors EXCEPTION;
PRAGMA EXCEPTION_INIT(errors, -24381); -- Error code for bulk operation exceptions
```

```
BEGIN
-- Perform bulk insert on the EMPLOYEES table for each employee in the employees table
FORALL i IN employees.FIRST .. employees.LAST SAVE EXCEPTIONS
    INSERT INTO EMPLOYEES (FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER,
HIRE_DATE, DEPARTMENT_ID, ROLE_ID, SALARY, BONUS)
    VALUES (employees(i).FIRST_NAME, employees(i).LAST_NAME, employees(i).EMAIL,
employees(i).PHONE_NUMBER,
employees(i).HIRE_DATE, employees(i).DEPARTMENT_ID, employees(i).ROLE_ID,
employees(i).SALARY, employees(i).BONUS);

    DBMS_OUTPUT.PUT_LINE('Total employees inserted: ' || SQL%ROWCOUNT);

EXCEPTION
    WHEN errors THEN
        -- Loop through each error in the SAVE EXCEPTIONS
        FOR i IN 1 .. SQL%BULK_EXCEPTIONS.COUNT LOOP
            DBMS_OUTPUT.PUT_LINE('Error at record ' || SQL%BULK_EXCEPTIONS(i).ERROR_INDEX
||
                                ': Error Code - ' || SQL%BULK_EXCEPTIONS(i).ERROR_CODE);
        END LOOP;
END;
/
```

Explanation of Code:

Record and Table Types: EmpRecord holds individual employee details, and EmpList stores multiple records for bulk processing.

FORALL with SAVE EXCEPTIONS: Allows bulk insertion; any error during insertion is saved, and processing continues with remaining records.

Exception Handling: SQL%BULK_EXCEPTIONS stores details of each error, looping through to display each error's index and code.

Total Count: SQL%ROWCOUNT outputs the count of successfully inserted rows.

Notes: SQL%BULK_EXCEPTIONS.COUNT provides the total error count, while ERROR_INDEX identifies each failed record for debugging.

Explicit Cursor

An explicit cursor in PL/SQL is a named cursor created by **the programmer to manage and control a query's context independently**. Unlike implicit cursors, which are automatically created by PL/SQL for single SQL statements, explicit cursors allow more control over multi-row queries and give access to each row's values in a controlled manner.

How Explicit Cursors Work

Explicit cursors are defined in three main steps:

Declare the cursor to define the SQL query.

Open the cursor to execute the query and initialize the result set.

Fetch rows one by one or in bulk from the cursor's result set.

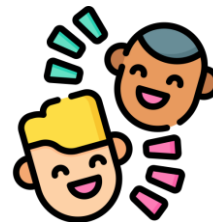
Close the cursor when done to release resources.

Key Benefits of Using Explicit Cursors

Row-by-Row Processing: Allows for processing rows one at a time in a controlled loop.

Flexible Error Handling: You can control what happens if no rows are returned or other exceptions occur.

Reusability: Cursors can be opened and fetched multiple times if needed (after closing them first).



Example of Explicit Cursor

```
1. DECLARE
2. CURSOR guru99_det IS SELECT emp_name FROM emp;
3. lv_emp_name emp.emp_name%type;
4. BEGIN
5. OPEN guru99_det;
6. LOOP
7. FETCH guru99_det INTO lv_emp_name;
8. IF guru99_det%NOTFOUND
9. THEN
10. EXIT;
11. END IF;
12. Dbms_output.put_line ('Employee Fetched: ' || lv_emp_name);
13. END LOOP;
14. Dbms_output.put_line ('Total rows fetched is ' || guru99_det%ROWCOUNT);
15. CLOSE guru99_det;
16. END;
17. /
```

Annotations:

- Cursor declaration (lines 2-3)
- opening cursor (line 5)
- Loop to Fetch cursor Data (lines 6-13)
- closing cursor (line 15)

Output:

```
Employee Fetched: BBB
Employee Fetched: XXX
Employee Fetched: YYY
Total rows fetched is 3
```



Source Image: <https://www.guru99.com/pl-sql-cursor.html>

Explicit Cursor – Example

1. `cursor%ISOPEN`
2. `cursor%FOUND`
3. `cursor%NOTFOUND`

```
DECLARE
-- Step 1: Declare the cursor with a SELECT statement
CURSOR emp_cursor IS
    SELECT employee_id, first_name, last_name
    FROM employees
    WHERE department_id = 10;

-- Variables to hold data for each fetched row
emp_id employees.employee_id%TYPE;
emp_first_name employees.first_name%TYPE;
emp_last_name employees.last_name%TYPE;

BEGIN
-- Step 2: Open the cursor and check if it's open
OPEN emp_cursor;
IF emp_cursor%ISOPEN THEN
    DBMS_OUTPUT.PUT_LINE('Cursor is now open.');
```

END IF;

-- Step 3: Fetch rows using a loop, applying cursor attributes

```
LOOP
    FETCH emp_cursor INTO emp_id, emp_first_name, emp_last_name;

    -- Check if a row was fetched
    IF emp_cursor%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Fetched row ' || emp_cursor%ROWCOUNT ||
            ': ' || emp_id || ', ' || emp_first_name || ' ' || emp_last_name);

    END IF;

    -- Exit loop if no rows are left
    EXIT WHEN emp_cursor%NOTFOUND;
END LOOP;

-- Step 4: Close the cursor and confirm it is closed
CLOSE emp_cursor;
IF NOT emp_cursor%ISOPEN THEN
    DBMS_OUTPUT.PUT_LINE('Cursor is now closed.');
```

END IF;

```
END;
/
```

Explicit Cursor - Example

```
DECLARE
-- Declare a cursor to select employee details for a specific department
CURSOR emp_cursor IS
    SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME, DEPARTMENT_ID
    FROM EMPLOYEES
    WHERE DEPARTMENT_ID = 10; -- Specify the department

-- Variables to hold the fetched data
v_employee_id EMPLOYEES.EMPLOYEE_ID%TYPE;
v_first_name EMPLOYEES.FIRST_NAME%TYPE;
v_last_name EMPLOYEES.LAST_NAME%TYPE;
v_department_id EMPLOYEES.DEPARTMENT_ID%TYPE;

BEGIN
-- Open the cursor
OPEN emp_cursor;

-- Fetch each row and process the data
LOOP
    -- Fetch data into variables
    FETCH emp_cursor INTO v_employee_id, v_first_name, v_last_name, v_department_id;

    -- Exit the loop when no more rows are returned
    EXIT WHEN emp_cursor%NOTFOUND;

    -- Display employee details
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_employee_id || ', Name: ' || v_first_name || ' ' ||
v_last_name || ', Department: ' || v_department_id);
END LOOP;
-- Close the cursor
CLOSE emp_cursor;

END;
/
```

Explanation of the Code

- 1. Cursor Declaration:** Defines emp_cursor to fetch details of employees in a specific department (DEPARTMENT_ID = 10).
- 2. Open Cursor:** The cursor is opened to begin fetching data.
- 3. Fetch and Loop:** Each row is fetched using FETCH emp_cursor INTO ... inside a loop. The loop continues until all rows are processed.
- 4. Display Data:** The fetched data is output for each employee.
- 5. Close Cursor:** The cursor is closed after the loop completes.

Notes

- This example uses %NOTFOUND to exit the loop when all rows have been processed.
- Explicit cursors are useful for row-by-row processing when multiple records need to be handled individually.

Explicit Cursor in Procedure— Multiple Rows %NOTFOUND

Explanation of the Procedure

1. **Cursor Declaration:** dept_cursor selects the department names from the departments table.
2. **Variable Declaration:** v_department_name holds each fetched department name.
3. **Opening the Cursor:** The cursor is opened to initiate the fetching of rows.
4. **Looping and Fetching Rows:** Each department name is fetched, printed, and the loop exits when no more rows are available.
5. **Closing the Cursor:** Once all rows are processed, the cursor is closed to release resources.

```
CREATE OR REPLACE PROCEDURE display_departments AS
-- Declare the cursor for retrieving department names
CURSOR dept_cursor IS
    SELECT department_name FROM departments;
-- Variable to hold each department name fetched by the cursor
v_department_name departments.department_name%TYPE;
BEGIN
-- Open the cursor
OPEN dept_cursor;

-- Loop through each row fetched by the cursor
LOOP
    FETCH dept_cursor INTO v_department_name;
    -- Exit the loop if no more rows are fetched
    EXIT WHEN dept_cursor%NOTFOUND;

    -- Output the department name
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_department_name);
END LOOP;

-- Close the cursor after processing
CLOSE dept_cursor;
END display_departments;
/
```

```
--this will display all department names in the departments table, printed one by one.
SET SERVEROUTPUT ON;
EXEC display_departments;
```

Call Procedure display_departments

Explicit Cursor in Procedure— Multiple Rows %NOTFOUND

Explanation of the Procedure

1. **Cursor Declaration:** dept_cursor selects the department names from the departments table.
2. **Variable Declaration:** v_department_name holds each fetched department name.
3. **Opening the Cursor:** The cursor is opened to initiate the fetching of rows.
4. **Looping and Fetching Rows:** Each department name is fetched, printed, and the loop exits when no more rows are available.
5. **Closing the Cursor:** Once all rows are processed, the cursor is closed to release resources.

```
CREATE OR REPLACE PROCEDURE display_departments AS
-- Declare the cursor for retrieving department names
CURSOR dept_cursor IS
    SELECT department_name FROM departments;
-- Variable to hold each department name fetched by the cursor
v_department_name departments.department_name%TYPE;
BEGIN
-- Open the cursor
OPEN dept_cursor;

-- Loop through each row fetched by the cursor
LOOP
    FETCH dept_cursor INTO v_department_name;
    -- Exit the loop if no more rows are fetched
    EXIT WHEN dept_cursor%NOTFOUND;

    -- Output the department name
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_department_name);
END LOOP;

-- Close the cursor after processing
CLOSE dept_cursor;
END display_departments;
/
```

```
--this will display all department names in the departments table, printed one by one.
SET SERVEROUTPUT ON;
EXEC display_departments;
```

Call Procedure display_departments

Explicit Cursor – Multiple Rows %FOUND

Key Points:

- **Explicit Cursor Declaration:** emp_cursor is explicitly declared, making it an explicit cursor.
- **Cursor FOR Loop:** By iterating over emp_cursor with a FOR loop, PL/SQL takes care of opening, fetching each row into emp_record, and closing emp_cursor once all rows are processed.
- **Automatic Management:** The FOR loop eliminates the need to write OPEN, FETCH, and CLOSE statements, simplifying the code.

```
SET SERVEROUTPUT ON SIZE 100000;

DECLARE
    -- Define the cursor to retrieve employees from department 1
    CURSOR emp_cursor IS
        SELECT employee_id, first_name, last_name
        FROM employees
        WHERE department_id = 1 AND ROWNUM <= 2; -- Adjust the condition as needed

BEGIN
    -- Loop through each row fetched by the cursor
    FOR emp_record IN emp_cursor LOOP
        -- Output the employee details
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.employee_id ||
                               ', Name: ' || emp_record.first_name || ' ' || emp_record.last_name);
    END LOOP;

    -- Check if any rows were fetched by the cursor using %FOUND
    IF emp_cursor%FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employees found in department 1.');
```

Parameterized Explicit Cursors

Breakdown of Comments:

- **Cursor Declaration:** The emp_cursor is declared with a parameter p_department_id, which allows filtering employees by department when the cursor is opened.
- **Fetching Data:** Inside the loop, rows are fetched from the cursor and stored in the emp_record. The loop will continue until no rows are left to fetch (emp_cursor%NOTFOUND).
- **Output:** The employee details (ID, first name, and last name) are displayed using DBMS_OUTPUT.PUT_LINE.
- **Cursor Closing:** The cursor is closed after all operations are completed to free resources.

These comments help explain the steps in opening the cursor, fetching rows, processing the data, and closing the cursor at the end.

```
DECLARE
-- Declare a cursor with a parameter for department ID
CURSOR emp_cursor (p_department_id NUMBER) IS
-- The SQL query that retrieves employee details based on department ID
SELECT employee_id, first_name, last_name
FROM employees
WHERE department_id = p_department_id;

-- Declare a record variable that will hold each row fetched from the cursor
emp_record emp_cursor%ROWTYPE;
BEGIN
-- Open the cursor and pass the department ID (in this case, department ID
10)
OPEN emp_cursor(10); -- Passing department ID as parameter

-- Start a loop to fetch each row from the cursor and process it
LOOP
-- Fetch the current row into the emp_record variable
FETCH emp_cursor INTO emp_record;

-- Exit the loop when there are no more rows to fetch
EXIT WHEN emp_cursor%NOTFOUND;

-- Output the employee details (employee ID, first name, and last name)
DBMS_OUTPUT.PUT_LINE(emp_record.employee_id || ' ' ||
emp_record.first_name || ' ' || emp_record.last_name);
END LOOP;

-- Close the cursor after all rows have been fetched and processed
CLOSE emp_cursor;
END;
```

Parameterized cursor

Parameterize Cursor

```
DECLARE
-- Define a parameterized cursor that accepts a department ID as input
CURSOR emp_cursor (p_department_id NUMBER) IS
    SELECT e.employee_id, e.first_name, e.last_name, e.email, e.salary, r.role_name,
    d.department_name
    FROM employees e
    JOIN roles r ON e.role_id = r.role_id
    JOIN departments d ON e.department_id = d.department_id
    WHERE e.department_id = p_department_id; -- Filter based on the given department ID

-- Variables to store employee details
v_employee_id employees.employee_id%TYPE;
v_first_name employees.first_name%TYPE;
v_last_name employees.last_name%TYPE;
v_email employees.email%TYPE;
v_salary employees.salary%TYPE;
v_role_name roles.role_name%TYPE;
v_department_name departments.department_name%TYPE;

BEGIN
-- Open the cursor with a specific department ID, e.g., department 10
OPEN emp_cursor(10);

LOOP
-- Fetch each row into the variables
    FETCH emp_cursor INTO v_employee_id, v_first_name, v_last_name, v_email, v_salary,
    v_role_name, v_department_name;

-- Exit when no more rows are returned
    EXIT WHEN emp_cursor%NOTFOUND;

-- Display employee and department details
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_employee_id);
    DBMS_OUTPUT.PUT_LINE('Name: ' || v_first_name || ' ' || v_last_name);
    DBMS_OUTPUT.PUT_LINE('Email: ' || v_email);
    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
    DBMS_OUTPUT.PUT_LINE('Role: ' || v_role_name);
    DBMS_OUTPUT.PUT_LINE('Department: ' || v_department_name);
    DBMS_OUTPUT.PUT_LINE('-----');
END LOOP;
-- Close the cursor after processing
CLOSE emp_cursor;
END;
/
```


Explicit Cursor - DBMS_SQL cursor Handle

The **DBMS_SQL cursor handle** is a unique identifier for a cursor that is created and managed using Oracle's DBMS_SQL package. Unlike regular PL/SQL cursors, which are typically static, the DBMS_SQL package provides a more flexible way to work with dynamic SQL statements (queries built and executed at runtime). Here's a breakdown of how the cursor handle works in this context:

1. Opening the Cursor:

- The cursor is opened with `DBMS_SQL.OPEN_CURSOR`, which returns a numeric handle (an integer) representing the cursor. This handle allows subsequent operations, such as parsing and execution, to be performed on that specific cursor instance.

```
v_cursor := DBMS_SQL.OPEN_CURSOR;
```

2. Using the Handle:

- The cursor handle (`v_cursor` in the example) is then used with various DBMS_SQL procedures and functions to parse the SQL statement, bind variables, define columns, execute the query, and fetch results.

```
DBMS_SQL.PARSE(v_cursor, 'SELECT * FROM employees WHERE department_id = :dept_id', DBMS_SQL.NATIVE);  
DBMS_SQL.BIND_VARIABLE(v_cursor, ':dept_id', 3);  
v_status := DBMS_SQL.EXECUTE(v_cursor);
```

3. Fetching Results:

After executing the SQL, the cursor handle is used to retrieve rows and column values. This involves defining columns with `DBMS_SQL.DEFINE_COLUMN` and then using `DBMS_SQL.FETCH_ROWS` and `DBMS_SQL.COLUMN_VALUE` to extract data for each row.

4. Closing the Cursor:

It's essential to close the `DBMS_SQL.CLOSE_CURSOR(v_cursor)`; e cursor with `DBMS_SQL.CLOSE_CURSOR` to release the resources allocated to it. Failure to do so may lead to resource leaks.

Error Handling:

- In case of an error, it's crucial to check if the cursor is still open using `DBMS_SQL.IS_OPEN(v_cursor)` and close it if necessary within the exception handling block.

Example Summary

The cursor handle enables flexible SQL execution and management, especially for queries that can't be pre-defined at compile time, such as those involving user inputs or conditional query building.

Explicit Cursor - DBMS_SQL cursor Handle – Example I

```
SET SERVEROUTPUT ON;

DECLARE
    v_cursor    INT; -- Cursor handle
    v_emp_id    EMPLOYEES.EMPLOYEE_ID%TYPE;
    v_first_name EMPLOYEES.FIRST_NAME%TYPE;
    v_last_name EMPLOYEES.LAST_NAME%TYPE;
    v_status    INT; -- To store the status of the cursor execution
    v_row_count INT; -- To store the row count after fetching

BEGIN
    -- Open a cursor for a dynamic SQL query
    v_cursor := DBMS_SQL.OPEN_CURSOR;

    -- Parse the dynamic SQL (query string)
    DBMS_SQL.PARSE(v_cursor, 'SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME
FROM EMPLOYEES WHERE DEPARTMENT_ID = :dept_id', DBMS_SQL.NATIVE);

    -- Bind the variable (department_id) to the placeholder in the query
    DBMS_SQL.BIND_VARIABLE(v_cursor, ':dept_id', 3);

    -- Define columns with specific data types to match the SELECT statement
    columns
    DBMS_SQL.DEFINE_COLUMN(v_cursor, 1, v_emp_id);      -- Numeric type for
EMPLOYEE_ID
    DBMS_SQL.DEFINE_COLUMN(v_cursor, 2, v_first_name, 50); -- VARCHAR2 type
with length 50 for FIRST_NAME
    DBMS_SQL.DEFINE_COLUMN(v_cursor, 3, v_last_name, 50); -- VARCHAR2
type with length 50 for LAST_NAME
```

```
-- Execute the query
v_status := DBMS_SQL.EXECUTE(v_cursor);
DBMS_OUTPUT.PUT_LINE('Execution status: ' || v_status);

-- Fetch and output each row's data until no rows remain
LOOP
    v_row_count := DBMS_SQL.FETCH_ROWS(v_cursor);
    EXIT WHEN v_row_count = 0;

    -- Retrieve and output each column value for the fetched row
    DBMS_SQL.COLUMN_VALUE(v_cursor, 1, v_emp_id);    -- First column
(EMPLOYEE_ID)
    DBMS_SQL.COLUMN_VALUE(v_cursor, 2, v_first_name); -- Second column
(FIRST_NAME)
    DBMS_SQL.COLUMN_VALUE(v_cursor, 3, v_last_name);  -- Third column
(LAST_NAME)

    -- Output the fetched data
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_emp_id || ', Name: ' ||
v_first_name || ' ' || v_last_name);
END LOOP;

-- Close the cursor after use
DBMS_SQL.CLOSE_CURSOR(v_cursor);

EXCEPTION
    WHEN OTHERS THEN
        -- Handle any exceptions and close the cursor if still open
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
        IF DBMS_SQL.IS_OPEN(v_cursor) THEN
            DBMS_SQL.CLOSE_CURSOR(v_cursor);
        END IF;
END;
/
```

Explicit Cursor - DBMS_SQL cursor Handle – Example II

```
CREATE OR REPLACE PROCEDURE GET_ALLOWANCES (p_is_applicable CHAR)
IS
    -- Explicit cursor for the ALLOWANCES table
    CURSOR allowance_cursor IS
        SELECT ALLOWANCE_ID, ROLE_ID, ALLOWANCE_AMOUNT
        FROM ALLOWANCES
        WHERE IS_APPLICABLE = p_is_applicable;

    -- DBMS_SQL cursor handle
    v_cursor      INT; -- Cursor handle for dynamic SQL
    v_allowance_id ALLOWANCES.ALLOWANCE_ID%TYPE;
    v_role_id     ALLOWANCES.ROLE_ID%TYPE;
    v_allowance_amount ALLOWANCES.ALLOWANCE_AMOUNT%TYPE;
    v_status      INT; -- To store the status of the cursor execution
    v_row_count   INT; -- To store the row count after fetching

BEGIN
    DBMS_OUTPUT.PUT_LINE('Using Explicit Cursor:');

    -- Using explicit cursor to fetch allowance records
    FOR rec IN allowance_cursor LOOP
        DBMS_OUTPUT.PUT_LINE('Allowance ID: ' || rec.ALLOWANCE_ID ||
                              ', Role ID: ' || rec.ROLE_ID ||
                              ', Allowance Amount: ' || rec.ALLOWANCE_AMOUNT);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Using DBMS_SQL Cursor:');

    -- Using DBMS_SQL cursor handle to fetch allowance records dynamically
    v_cursor := DBMS_SQL.OPEN_CURSOR;

    -- Parse the dynamic SQL query
    DBMS_SQL.PARSE(v_cursor, 'SELECT ALLOWANCE_ID, ROLE_ID, ALLOWANCE_AMOUNT FROM
ALLOWANCES WHERE IS_APPLICABLE = :is_applicable', DBMS_SQL.NATIVE);
```

```
-- Bind variable for dynamic filtering
DBMS_SQL.BIND_VARIABLE(v_cursor, 'is_applicable', p_is_applicable);

-- Define columns for retrieval
DBMS_SQL.DEFINE_COLUMN(v_cursor, 1, v_allowance_id); -- ALLOWANCE_ID
DBMS_SQL.DEFINE_COLUMN(v_cursor, 2, v_role_id);      -- ROLE_ID
DBMS_SQL.DEFINE_COLUMN(v_cursor, 3, v_allowance_amount); -- ALLOWANCE_AMOUNT

-- Execute the cursor and fetch rows in a loop
v_status := DBMS_SQL.EXECUTE(v_cursor);

LOOP
    v_row_count := DBMS_SQL.FETCH_ROWS(v_cursor);
    EXIT WHEN v_row_count = 0;

    -- Get values for each column in the current row
    DBMS_SQL.COLUMN_VALUE(v_cursor, 1, v_allowance_id);
    DBMS_SQL.COLUMN_VALUE(v_cursor, 2, v_role_id);
    DBMS_SQL.COLUMN_VALUE(v_cursor, 3, v_allowance_amount);

    -- Output the fetched data
    DBMS_OUTPUT.PUT_LINE('Allowance ID: ' || v_allowance_id ||
                          ', Role ID: ' || v_role_id ||
                          ', Allowance Amount: ' || v_allowance_amount);

END LOOP;
-- Close the DBMS_SQL cursor
DBMS_SQL.CLOSE_CURSOR(v_cursor);
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
        IF DBMS_SQL.IS_OPEN(v_cursor) THEN
            DBMS_SQL.CLOSE_CURSOR(v_cursor);
        END IF;
END GET_ALLOWANCES;
/

-- To execute the procedure:
EXEC GET_ALLOWANCES('Y');
```

Using Cursors and Functions to Fetch and Process Employee Data

```
-- Function to fetch employee data and return a SYS_REFCURSOR
CREATE OR REPLACE FUNCTION get_all_employee_data
RETURN SYS_REFCURSOR
IS
    -- Declare a cursor variable to hold the result set
    emp_cursor SYS_REFCURSOR;
BEGIN
    -- Open the cursor for a SELECT query to fetch specific columns from the 'employees' table
    OPEN emp_cursor FOR
        SELECT employee_id, first_name, bonus, salary FROM employees;

    -- Return the cursor so it can be used in the calling block
    RETURN emp_cursor;
END;
/

-- Anonymous PL/SQL block to call the function and process the employee data
DECLARE
    -- Declare a cursor variable to hold the result of the function call
    c_emp_data SYS_REFCURSOR;

    -- Declare variables to hold the values fetched from the cursor (using %TYPE for data type consistency)
    emp_id     employees.employee_id%TYPE; -- Variable to store employee ID (data type from employees table)
    emp_name   employees.first_name%TYPE;  -- Variable to store employee name (data type from employees table)
    emp_bonus  employees.bonus%TYPE;       -- Variable to store employee bonus (data type from employees table)
    emp_salary employees.salary%TYPE;      -- Variable to store employee salary (data type from employees table)
BEGIN
    -- Call the function to get the cursor containing employee data
    c_emp_data := get_all_employee_data;

    -- Fetch each row from the cursor and process it
    LOOP
        -- Fetch the current row into the declared variables
        FETCH c_emp_data INTO emp_id, emp_name, emp_bonus, emp_salary;

        -- Exit the loop when no more rows are found
        EXIT WHEN c_emp_data%NOTFOUND;

        -- Process each record, here we output the data using DBMS_OUTPUT
        DBMS_OUTPUT.PUT_LINE('ID: ' || emp_id || ', Name: ' || emp_name ||
                               ', Bonus: ' || emp_bonus || ', Salary: ' || emp_salary);
    END LOOP;

    -- Close the cursor once all rows have been processed
    CLOSE c_emp_data;
END;
/
```

Explicit Cursor FOR Loops I

Using a Cursor to Fetch
Employees by Department

```
SET SERVEROUTPUT ON SIZE 100000;
```

```
DECLARE
```

```
-- Cursor to fetch employee details from department_id = 30
```

```
CURSOR c_emp_cursor IS
```

```
    SELECT employee_id, last_name, first_name, email, phone_number, hire_date
```

```
    FROM employees
```

```
    WHERE department_id = 30;
```

```
-- Custom exception to handle the case where no employees are found
```

```
e_no_employees_found EXCEPTION;
```

```
BEGIN
```

```
-- Initialize a counter to track if the loop finds any records
```

```
DECLARE
```

```
    emp_found BOOLEAN := FALSE; -- Flag to check if any employee is found
```

```
BEGIN
```

```
-- Loop through each record returned by the cursor
```

```
FOR emp_record IN c_emp_cursor LOOP
```

```
-- Set the flag to TRUE if at least one record is found
```

```
    emp_found := TRUE;
```

```
-- Output the employee details
```

```
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.employee_id ||
```

```
                           ' Name: ' || emp_record.first_name || ' ' || emp_record.last_name ||
```

```
                           ' Email: ' || emp_record.email ||
```

```
                           ' Phone: ' || emp_record.phone_number ||
```

```
                           ' Hire Date: ' || TO_CHAR(emp_record.hire_date, 'YYYY-MM-DD')));
```

```
END LOOP;
```

```
-- Check if no employees were found and raise an exception
```

```
IF NOT emp_found THEN
```

```
    RAISE e_no_employees_found;
```

```
END IF;
```

```
END;
```

```
EXCEPTION
```

```
    WHEN e_no_employees_found THEN
```

```
        DBMS_OUTPUT.PUT_LINE('No employees found in department ID: 30');
```

```
    WHEN OTHERS THEN
```

```
-- Handle any unexpected errors
```

```
    DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
```

```
END;
```

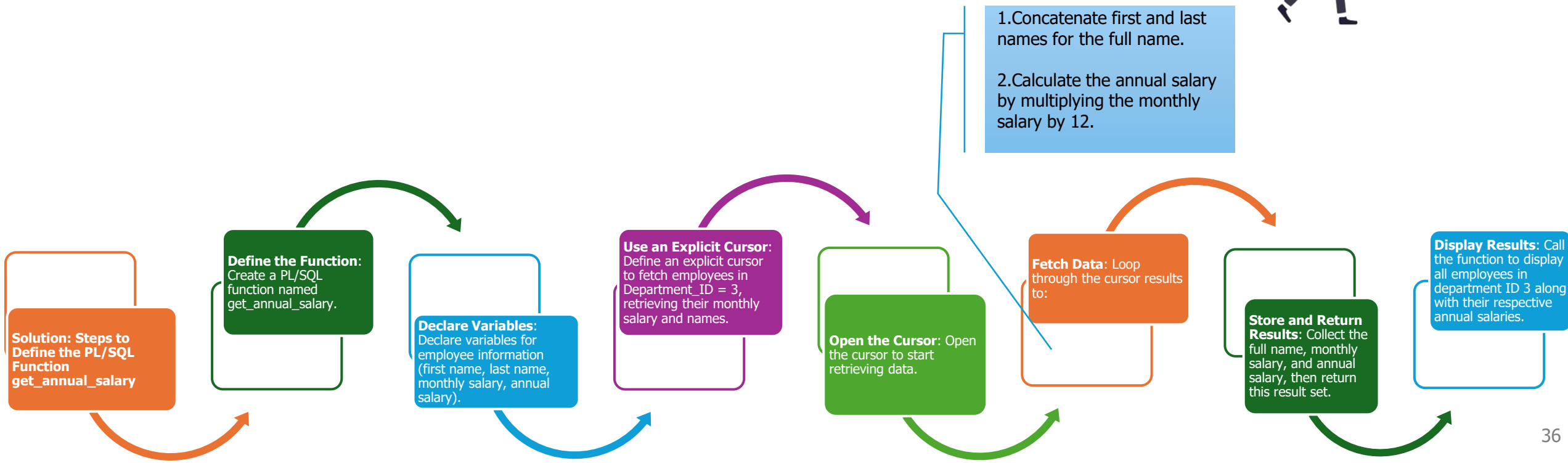
```
/
```

Exercise – Explicitly Cursor with PLSQL

Question: Write a PL/SQL function that calculates the annual salary of each employee in the department with Department_ID = 3. The function should return a result set containing the following information for each employee:

- Concatenated full name (first name and last name)
- Monthly salary
- Calculated annual salary

After creating the function, display the output by listing all employees in department ID 3 with their respective annual salaries.



Cursor – Create function

Create the Function

--Step 1: Create the Function with Comments

```
CREATE OR REPLACE FUNCTION get_annual_salary (p_department_id NUMBER)
RETURN SYS_REFCURSOR
IS
    -- Declare a cursor variable to return the result set
    annual_salary_cursor SYS_REFCURSOR;
BEGIN
    -- Open a cursor for the result set containing employee's full name, monthly salary, and
    -- calculated annual salary
    OPEN annual_salary_cursor FOR
        SELECT first_name || ' ' || last_name AS full_name, -- Concatenate first name and last
        name
            salary AS monthly_salary, -- Select the monthly salary and alias it
            salary * 12 AS annual_salary -- Calculate annual salary by multiplying
            monthly salary by 12
        FROM employees -- From the employees table
        WHERE department_id = p_department_id; -- Filter results by the provided
        department ID

    -- Return the cursor containing the result set
    RETURN annual_salary_cursor;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employees found for the provided department ID.');
```

```
    RETURN NULL;
```

```
    WHEN OTHERS THEN
```

```
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
```

```
    RETURN NULL;
```

```
END;
```

```
/
```


Call the function

Call the Function and
Display the Results

```
DECLARE
  -- Declare a cursor variable to hold the result set
  emp_salary_cursor SYS_REFCURSOR;

  -- Declare variables to store each column of the result set
  v_full_name VARCHAR2(100);
  v_monthly_salary NUMBER;
  v_annual_salary NUMBER;
BEGIN
  -- Call the function and assign the returned cursor to emp_salary_cursor
  emp_salary_cursor := get_annual_salary(10); -- Replace 10 with the desired department ID

  -- Loop through each row in the cursor
  LOOP
    -- Fetch each row into the variables
    FETCH emp_salary_cursor INTO v_full_name, v_monthly_salary, v_annual_salary;

    -- Exit the loop when no more rows are found
    EXIT WHEN emp_salary_cursor%NOTFOUND;

    -- Display the fetched values
    DBMS_OUTPUT.PUT_LINE('Full Name: ' || v_full_name);
    DBMS_OUTPUT.PUT_LINE('Monthly Salary: ' || v_monthly_salary);
    DBMS_OUTPUT.PUT_LINE('Annual Salary: ' || v_annual_salary);
    DBMS_OUTPUT.PUT_LINE('-----');
  END LOOP;

  -- Close the cursor
  CLOSE emp_salary_cursor;

EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END;
/
```

Question: Explicit Cursor, Type Object, Explicit Exception

Question:

Write the PL/SQL code to implement the `get_annual_salary` function according to the above requirements. Also, write an anonymous block to call this function with a department ID, display the result, and handle any exceptions that may occur during the function execution.

Follow-up Questions:

1. Why is an explicit cursor used in this function instead of an implicit one?
2. What advantages does returning an `emp_salary_table` object type provide in terms of data handling and readability?
3. Describe how each exception (`NO_DATA_FOUND`, `INVALID_CURSOR`, `OTHERS`) is handled and explain why such detailed exception handling is necessary in PL/SQL functions.

```
-- Create an object type to represent individual employee salary details
CREATE OR REPLACE TYPE emp_salary_obj AS OBJECT (
    full_name    VARCHAR2(100), -- Full name of the employee (first name and last name
concatenated)
    monthly_salary NUMBER,      -- Monthly salary of the employee
    annual_salary NUMBER        -- Calculated annual salary (monthly salary multiplied by 12)
);
/

-- Create a table type to hold multiple emp_salary_obj objects
CREATE OR REPLACE TYPE emp_salary_table AS TABLE OF emp_salary_obj;
/
```

Step 1: Define the Object and Table Types (if not already created)

Solution I – Loop

Solution

```
CREATE OR REPLACE FUNCTION get_annual_salary (p_department_id NUMBER)
RETURN emp_salary_table
IS
-- Declare a variable of the table type to store results
emp_salaries emp_salary_table := emp_salary_table();
-- Declare an explicit cursor for fetching employee data
CURSOR emp_cursor IS
    SELECT first_name || ' ' || last_name AS full_name,
           salary AS monthly_salary,
           salary * 12 AS annual_salary
    FROM employees
    WHERE department_id = p_department_id;

-- Variables to store individual column values
v_full_name    VARCHAR2(100);
v_monthly_salary NUMBER;
v_annual_salary NUMBER;
BEGIN
-- Open the cursor
OPEN emp_cursor;
-- Fetch each row from the cursor and populate the emp_salaries table
LOOP
    FETCH emp_cursor INTO v_full_name, v_monthly_salary, v_annual_salary;
    -- Exit when there are no more rows to fetch
    EXIT WHEN emp_cursor%NOTFOUND;

    -- Add each fetched row as an object to the emp_salaries table
    emp_salaries.EXTEND;
    emp_salaries(emp_salaries.COUNT) := emp_salary_obj(v_full_name, v_monthly_salary, v_annual_salary);
END LOOP;
-- Close the cursor
CLOSE emp_cursor;
-- Return the populated emp_salaries table
RETURN emp_salaries;

EXCEPTION
WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('No employees found for the provided department ID.');
```

```
    RETURN emp_salary_table(); -- Return an empty table if no data is found
WHEN INVALID_CURSOR THEN
    DBMS_OUTPUT.PUT_LINE('Error: Attempted to operate on an unopened or closed cursor.');
```

```
    RETURN emp_salary_table(); -- Return an empty table if there's an invalid cursor operation
WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
    RETURN emp_salary_table(); -- Return an empty table for any other errors
END;
/
```

Solution II – Bulk Collection

Solution II – Bulk Collection

```
CREATE OR REPLACE FUNCTION get_annual_salary (p_department_id NUMBER)
RETURN emp_salary_table
IS
    -- Declare a variable of the table type to store results
    emp_salaries emp_salary_table := emp_salary_table();

    -- Declare an explicit cursor for fetching employee data
    CURSOR emp_cursor IS
        SELECT first_name || ' ' || last_name AS full_name, -- Concatenate first and last name
               salary AS monthly_salary,                  -- Select monthly salary
               salary * 12 AS annual_salary                -- Calculate annual salary
        FROM employees
        WHERE department_id = p_department_id;

BEGIN
    -- Fetch all rows from the cursor in bulk into the emp_salaries collection
    OPEN emp_cursor;
    FETCH emp_cursor BULK COLLECT INTO emp_salaries;
    CLOSE emp_cursor;

    -- Return the populated emp_salaries table
    RETURN emp_salaries;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employees found for the provided department ID.');
```

```
        RETURN emp_salary_table(); -- Return an empty table if no data is found
    WHEN INVALID_CURSOR THEN
        DBMS_OUTPUT.PUT_LINE('Error: Attempted to operate on an unopened or closed cursor.');
```

```
        RETURN emp_salary_table(); -- Return an empty table if there's an invalid cursor operation
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
        RETURN emp_salary_table(); -- Return an empty table for any other errors
END;
/
```

Additional Notes

- **Bulk Collection without Loop:** Using BULK COLLECT allows for the elimination of the explicit LOOP, making the function code shorter and more efficient.
- **Memory Considerations:** BULK COLLECT is suitable for moderate data volumes. For very large datasets, consider using LIMIT to fetch data in chunks to avoid memory issues.

Call the function

Call the function

```
DECLARE
  -- Variable to hold the returned emp_salary_table
  employee_salaries emp_salary_table;

BEGIN
  -- Call the function and assign the result to employee_salaries
  employee_salaries := get_annual_salary(10); -- Replace 10 with the desired department ID

  -- Loop through each employee record in the employee_salaries table
  FOR i IN 1 .. employee_salaries.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE('Full Name: ' || employee_salaries(i).full_name);
    DBMS_OUTPUT.PUT_LINE('Monthly Salary: ' || employee_salaries(i).monthly_salary);
    DBMS_OUTPUT.PUT_LINE('Annual Salary: ' || employee_salaries(i).annual_salary);
    DBMS_OUTPUT.PUT_LINE('-----');
  END LOOP;

EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('An error occurred while retrieving salaries: ' || SQLERRM);
END;
/
```

Explanation

1. **Explicit Cursor:** An explicit cursor, emp_cursor, is declared and used to fetch the data row by row.
2. **Looping and Extending the Table:** Inside the loop, emp_salaries.EXTEND is used to add a new row to the emp_salaries table for each employee record fetched.
3. **Exception Handling:**
 - NO_DATA_FOUND handles the case where no employees are found for the department.
 - INVALID_CURSOR handles any issues with cursor operations.
 - OTHERS handles any remaining, unexpected errors.

This approach provides more granular control over cursor usage and specific exceptions.

Question – About Using cursor in Package

You are tasked with writing a PL/SQL package to process employee data for a given department. The package includes a procedure that fetches employee information (*Employee ID, First Name, Last Name, and Salary*) from the EMPLOYEES table based on the department ID.

Your procedure uses an explicit cursor and handles exceptions like **NO_DATA_FOUND** and unexpected errors (OTHERS). The following behavior is expected:

1. The procedure should output the employee's details for each record fetched from the cursor.
2. If no employees are found in the specified department, a custom exception should be raised with an appropriate error message.
3. The procedure must handle any unexpected errors gracefully by outputting the error message.

Task:

- Explain the role of **v_row_count** in the procedure and how it helps detect if no records are fetched.
- What would happen if the **v_row_count** is 0 after the cursor loop? How does the custom exception handling help in this case?
- Describe the significance of the **NO_DATA_FOUND** and OTHERS exceptions in your procedure.
- What would be the output if the procedure is executed with a department ID that does not have any employees?



Solution – Package Specification (1/2)

1. Package Specification (emp_package_spec.sql)

```
CREATE OR REPLACE PACKAGE emp_package AS
  -- Declare an explicit cursor in the package specification
  CURSOR emp_cursor(p_dept_id IN NUMBER) IS
    SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME, SALARY
    FROM EMPLOYEES
    WHERE DEPARTMENT_ID = p_dept_id;

  -- Procedure to process the employee data
  PROCEDURE process_employee_data(p_dept_id IN NUMBER);
END emp_package;
/
```


Solution – Package Body (2/2)

2. Package Body with Exception Handling (emp_package_body.sql)

```
CREATE OR REPLACE PACKAGE BODY emp_package AS

    -- Procedure implementation to process the employee data
    PROCEDURE process_employee_data(p_dept_id IN NUMBER) IS
        -- Declare variables to hold data from the cursor
        v_emp_id    EMPLOYEES.EMPLOYEE_ID%TYPE;
        v_first_name EMPLOYEES.FIRST_NAME%TYPE;
        v_last_name  EMPLOYEES.LAST_NAME%TYPE;
        v_salary     EMPLOYEES.SALARY%TYPE;
        v_row_count  INT := 0; -- To track number of rows fetched

    BEGIN
        -- Open the explicit cursor and loop through each record
        FOR emp_record IN emp_cursor(p_dept_id) LOOP
            -- Fetch data from the cursor into variables
            v_emp_id := emp_record.EMPLOYEE_ID;
            v_first_name := emp_record.FIRST_NAME;
            v_last_name := emp_record.LAST_NAME;
            v_salary := emp_record.SALARY;

            -- Output employee details
            DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_emp_id ||
                                ', Name: ' || v_first_name || ' ' || v_last_name ||
                                ', Salary: ' || v_salary);

            -- Increment row count
            v_row_count := v_row_count + 1;
        END LOOP;

        -- If no employees found, raise a custom exception
        IF v_row_count = 0 THEN
            RAISE_APPLICATION_ERROR(-20001, 'No employees found in the specified department.');
        END IF;

    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('No employees found for department ID ' || p_dept_id);
        WHEN OTHERS THEN
            -- Log the error if something unexpected happens
            DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
            -- Optionally, re-raise the error if needed
            RAISE;
        END process_employee_data;

END emp_package;
/
```

Solution – Package Body (2/2)

```
-- Call the procedure from the package to process employees in department 10  
EXEC emp_package.process_employee_data(10);
```

Execute the Procedure

```
BEGIN  
    -- Call the procedure from the package  
    emp_package.process_employee_data(10);  
END;  
/
```

Using a PL/SQL Anonymous Block

```
BEGIN  
    -- Call the procedure  
    emp_package.process_employee_data(10);  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('No employees found in the department.');
```

Using a PL/SQL Anonymous Block with exception handling

```
BEGIN  
    -- Call the procedure using dynamic SQL  
    EXECUTE IMMEDIATE 'BEGIN emp_package.process_employee_data(10); END;';  
END;  
/
```

Using DBMS_SQL (Dynamic SQL)

If you need dynamic execution (for example, executing the procedure with different department IDs at runtime), you can use DBMS_SQL to call the package procedure dynamically.

Exercises

Question:

Given the ALLOWANCES table with the following structure:

Column	Data Type
ALLOWANCE_ID	NUMBER
ROLE_ID	NUMBER
ALLOWANCE_AMOUNT	NUMBER
EFFECTIVE_DATE	DATE
IS_APPLICABLE	CHAR
COMMENTS	VARCHAR2(255)

Write a PL/SQL block that declares a parameterized cursor named `allowance_cursor`, which accepts two optional parameters: `p_role_id` (with a default value of 1) and `p_is_applicable` (with a default value of 'Y'). The cursor should retrieve `ALLOWANCE_ID`, `ALLOWANCE_AMOUNT`, `EFFECTIVE_DATE`, and `COMMENTS` for the given `ROLE_ID` and `IS_APPLICABLE` status.



The block should:

1. Open the cursor with default parameters.
2. Display each record's `ALLOWANCE_ID`, `ALLOWANCE_AMOUNT`, `EFFECTIVE_DATE`, and `COMMENTS`.
3. Close the cursor.
4. Reopen the cursor with `ROLE_ID = 2` and `IS_APPLICABLE = 'N'`, display the records again, and close the cursor.

Sample Output Expected:

For each record, the output should look like:

Allowance ID: <ALLOWANCE_ID>
Amount: <ALLOWANCE_AMOUNT>
Effective Date: <EFFECTIVE_DATE>
Comments: <COMMENTS>



Solution – Cursor with Parameter

```
DECLARE
-- Define a parameterized cursor with default values for ROLE_ID and IS_APPLICABLE
CURSOR allowance_cursor (p_role_id NUMBER DEFAULT 1, p_is_applicable CHAR DEFAULT 'Y') IS
    SELECT allowance_id, allowance_amount, effective_date, comments
    FROM allowances
    WHERE role_id = p_role_id
    AND is_applicable = p_is_applicable; -- Apply additional filter for applicability

-- Variables to store allowance details
v_allowance_id allowances.allowance_id%TYPE;
v_allowance_amount allowances.allowance_amount%TYPE;
v_effective_date allowances.effective_date%TYPE;
v_comments allowances.comments%TYPE;
BEGIN
-- Open the cursor with default values
OPEN allowance_cursor;

-- Process each row returned by the cursor
LOOP
    FETCH allowance_cursor INTO v_allowance_id, v_allowance_amount, v_effective_date, v_comments;
    -- Exit loop when no more rows are found
    EXIT WHEN allowance_cursor%NOTFOUND;
    -- Output allowance details
    DBMS_OUTPUT.PUT_LINE('Allowance ID: ' || v_allowance_id);
    DBMS_OUTPUT.PUT_LINE('Amount: ' || v_allowance_amount);
    DBMS_OUTPUT.PUT_LINE('Effective Date: ' || TO_CHAR(v_effective_date, 'YYYY-MM-DD'));
    DBMS_OUTPUT.PUT_LINE('Comments: ' || NVL(v_comments, 'N/A'));
    DBMS_OUTPUT.PUT_LINE('-----');
END LOOP;
-- Close the cursor after processing
CLOSE allowance_cursor;
-- Now let's open the cursor with specific values for ROLE_ID and IS_APPLICABLE
DBMS_OUTPUT.PUT_LINE('Opening cursor with specific parameters (ROLE_ID = 2, IS_APPLICABLE = N)');
OPEN allowance_cursor(2, 'N');
-- Process each row for the specific parameters
LOOP
    FETCH allowance_cursor INTO v_allowance_id, v_allowance_amount, v_effective_date, v_comments;

    EXIT WHEN allowance_cursor%NOTFOUND;

    DBMS_OUTPUT.PUT_LINE('Allowance ID: ' || v_allowance_id);
    DBMS_OUTPUT.PUT_LINE('Amount: ' || v_allowance_amount);
    DBMS_OUTPUT.PUT_LINE('Effective Date: ' || TO_CHAR(v_effective_date, 'YYYY-MM-DD'));
    DBMS_OUTPUT.PUT_LINE('Comments: ' || NVL(v_comments, 'N/A'));
    DBMS_OUTPUT.PUT_LINE('-----');
END LOOP;
-- Close the cursor
CLOSE allowance_cursor;
END;
```

Explanation of key points

The line `DBMS_OUTPUT.PUT_LINE('Comments: ' || NVL(v_comments, 'N/A'));` in PL/SQL is used to display the COMMENTS column value from the ALLOWANCES table. Here's how it works:

- **DBMS_OUTPUT.PUT_LINE:** This procedure outputs text to the console in Oracle SQL Developer or similar tools where output can be displayed.
- **NVL(v_comments, 'N/A'):** The NVL function is used to handle NULL values. In this case:
 - If the COMMENTS column (v_comments) contains a NULL value, NVL will return 'N/A' instead.
 - If v_comments is not NULL, it will return the actual content of v_comments.

So, `DBMS_OUTPUT.PUT_LINE('Comments: ' || NVL(v_comments, 'N/A'));` ensures that:

- If the COMMENTS field has data, it displays that data.
- If the COMMENTS field is NULL, it displays N/A, indicating that there's no comment for that record.

Question II

Cursor with Parameter – Default Value

Part A:

- 1.What is the purpose of using a parameterized cursor in this code?
- 2.Describe the role of the default value `p_min_salary NUMBER DEFAULT 3000` in the cursor declaration.

Part B:

- 1.What will be displayed if the employees in **Department 10** do not meet the minimum salary criteria of **3000**?
- 2.If the cursor is executed with parameters `employee_cursor(20, 5000)`, what conditions will be applied to retrieve records?

Part C:

Modify the code to:

- 1.Add an additional filter for employees with hire dates **after January 1, 2020**.
- 2.Output only the employee ID, name, and hire date in the DBMS output.

Solution – Cursor with Parameter – Default values

```
DECLARE
```

```
-- Define a parameterized cursor to fetch employee details by department and minimum salary
```

```
CURSOR employee_cursor (p_department_id NUMBER, p_min_salary NUMBER DEFAULT 3000)
```

```
IS
```

```
    SELECT employee_id, first_name, last_name, hire_date, salary
```

```
    FROM employees
```

```
    WHERE department_id = p_department_id
```

```
          AND salary >= p_min_salary; -- Condition for minimum salary
```

```
BEGIN
```

```
-- Display employees for a specific department (e.g., department 10) with minimum salary of 3000
```

```
DBMS_OUTPUT.PUT_LINE('Employees in Department 10 with Salary >= 3000');
```

```
FOR emp_record IN employee_cursor(10) LOOP
```

```
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.employee_id);
```

```
    DBMS_OUTPUT.PUT_LINE('Name: ' || emp_record.first_name || ' ' || emp_record.last_name);
```

```
    DBMS_OUTPUT.PUT_LINE('Hire Date: ' || TO_CHAR(emp_record.hire_date, 'YYYY-MM-DD'));
```

```
    DBMS_OUTPUT.PUT_LINE('Salary: ' || emp_record.salary);
```

```
    DBMS_OUTPUT.PUT_LINE('-----');
```

```
END LOOP;
```

```
-- Display employees in department 20 with minimum salary of 5000
```

```
DBMS_OUTPUT.PUT_LINE('Employees in Department 20 with Salary >= 5000');
```

```
FOR emp_record IN employee_cursor(20, 5000) LOOP
```

```
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.employee_id);
```

```
    DBMS_OUTPUT.PUT_LINE('Name: ' || emp_record.first_name || ' ' || emp_record.last_name);
```

```
    DBMS_OUTPUT.PUT_LINE('Hire Date: ' || TO_CHAR(emp_record.hire_date, 'YYYY-MM-DD'));
```

```
    DBMS_OUTPUT.PUT_LINE('Salary: ' || emp_record.salary);
```

```
    DBMS_OUTPUT.PUT_LINE('-----');
```

```
END LOOP;
```

```
END;
```

```
/
```


Solution with Custom Exception Handling

```
DECLARE
    -- Define a parameterized cursor to fetch employee details by department and
    minimum salary
    CURSOR employee_cursor (p_department_id NUMBER, p_min_salary NUMBER
    DEFAULT 3000) IS
        SELECT employee_id, first_name, last_name, hire_date, salary
        FROM employees
        WHERE department_id = p_department_id
        AND salary >= p_min_salary; -- Condition for minimum salary

    -- Define a custom exception
    no_employees_found EXCEPTION;
    -- Variable to check if employees were found
    emp_count INTEGER := 0;

BEGIN
    -- Display employees for a specific department (e.g., department 10) with
    minimum salary of 3000
    DBMS_OUTPUT.PUT_LINE('Employees in Department 10 with Salary >= 3000');
    FOR emp_record IN employee_cursor(10) LOOP
        emp_count := emp_count + 1; -- Increment count if an employee is found
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.employee_id);
        DBMS_OUTPUT.PUT_LINE('Name: ' || emp_record.first_name || ' ' ||
        emp_record.last_name);
        DBMS_OUTPUT.PUT_LINE('Hire Date: ' || TO_CHAR(emp_record.hire_date, 'YYYY-
        MM-DD'));
        DBMS_OUTPUT.PUT_LINE('Salary: ' || emp_record.salary);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;
```

```
-- Raise custom exception if no employees were found in department 10
IF emp_count = 0 THEN
    RAISE no_employees_found;
END IF;

-- Reset emp_count for the next department
emp_count := 0;

-- Display employees in department 20 with minimum salary of 5000
DBMS_OUTPUT.PUT_LINE('Employees in Department 20 with Salary >= 5000');
FOR emp_record IN employee_cursor(20, 5000) LOOP
    emp_count := emp_count + 1; -- Increment count if an employee is found
    DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.employee_id);
    DBMS_OUTPUT.PUT_LINE('Name: ' || emp_record.first_name || ' ' ||
    emp_record.last_name);
    DBMS_OUTPUT.PUT_LINE('Hire Date: ' || TO_CHAR(emp_record.hire_date, 'YYYY-
    MM-DD'));
    DBMS_OUTPUT.PUT_LINE('Salary: ' || emp_record.salary);
    DBMS_OUTPUT.PUT_LINE('-----');
END LOOP;

-- Raise custom exception if no employees were found in department 20
IF emp_count = 0 THEN
    RAISE no_employees_found;
END IF;

EXCEPTION
    WHEN no_employees_found THEN
        DBMS_OUTPUT.PUT_LINE('No employees found for the specified department and
        minimum salary.');
```

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
```

```
/
```

Example: Cursor for Calculating Employee Bonuses

Let's say that each employee's bonus is calculated based on their ROLE_ID and SALARY. We can use a cursor to calculate bonuses for each employee and update their BONUS field accordingly.

First, create a procedure that uses a cursor to calculate and apply the bonus.

```
-- Procedure to calculate and update the bonus for each employee
CREATE OR REPLACE PROCEDURE calculate_employee_bonus IS

-- Cursor to fetch each employee's ID, role, and salary from EMPLOYEES table
CURSOR emp_cursor IS
  SELECT EMPLOYEE_ID, ROLE_ID, SALARY
  FROM EMPLOYEES;

-- Variable to store the calculated bonus for each employee
v_bonus NUMBER;

BEGIN
  -- Loop through each employee record fetched by the cursor
  FOR emp_rec IN emp_cursor LOOP

    -- Determine bonus based on employee role
    IF emp_rec.ROLE_ID = 1 THEN
      v_bonus := emp_rec.SALARY * 0.10; -- 10% bonus for role ID 1
    ELSIF emp_rec.ROLE_ID = 2 THEN
      v_bonus := emp_rec.SALARY * 0.15; -- 15% bonus for role ID 2
    ELSE
      v_bonus := emp_rec.SALARY * 0.05; -- 5% bonus for other roles
    END IF;

    -- Update the BONUS column in the EMPLOYEES table with the calculated bonus
    UPDATE EMPLOYEES
    SET BONUS = v_bonus
    WHERE EMPLOYEE_ID = emp_rec.EMPLOYEE_ID;

  END LOOP;

END;
/
```

In this example:

- The cursor emp_cursor retrieves each EMPLOYEE_ID, ROLE_ID, and SALARY from the EMPLOYEES table.
- For each employee, the v_bonus variable calculates the bonus based on the role.
- The bonus is then updated for each employee.

This trigger automatically records salary changes without requiring manual data entry.

Application of Cursors

Combining Triggers and Cursors: A Practical Scenario

A combined use of triggers and cursors can automate complex data manipulations. For example, when an employee's salary is updated, a trigger could automatically calculate their new bonus using a cursor that iterates through all affected records.

Example: Trigger with Cursor to Update Bonus on Salary Change

Here's an example where we use both a trigger and a cursor. When a salary is updated in the EMPLOYEES table, a trigger fires, and a cursor calculates a new bonus based on the updated salary.

In this example:

- The role_bonus_cursor retrieves the bonus percentage for each role.
- When a salary is updated, the trigger uses the cursor to find the relevant bonus percentage for the employee's role and updates their BONUS field accordingly.

```
-- Trigger to calculate and update the bonus whenever an employee's salary is updated
CREATE OR REPLACE TRIGGER calculate_bonus_on_salary_update
-- Trigger fires after an update on the SALARY column in the EMPLOYEES table
AFTER UPDATE OF SALARY ON EMPLOYEES
FOR EACH ROW -- Ensures the trigger fires for each row affected by the update
DECLARE
-- Cursor to retrieve bonus percentages based on role from the ROLE_BONUS_CONFIG table
CURSOR role_bonus_cursor IS
    SELECT ROLE_ID, BONUS_PERCENT
    FROM ROLE_BONUS_CONFIG; -- Hypothetical table that stores bonus percentage for each role

-- Variable to store the bonus percentage for the employee's role
v_bonus_percent NUMBER;
BEGIN
-- Loop through each role in the cursor to find the matching role for the updated employee
FOR role_rec IN role_bonus_cursor LOOP
-- If the employee's ROLE_ID matches the current role record, store the bonus percentage
IF :NEW.ROLE_ID = role_rec.ROLE_ID THEN
    v_bonus_percent := role_rec.BONUS_PERCENT;
    EXIT; -- Exit the loop once the matching role is found
END IF;
END LOOP;

-- Calculate the new bonus using the updated salary and bonus percentage, then update the employee record
UPDATE EMPLOYEES
SET BONUS = :NEW.SALARY * v_bonus_percent / 100
WHERE EMPLOYEE_ID = :NEW.EMPLOYEE_ID;
END;
/
```

Disadvantages of Explicit Cursors

Disadvantage	Explanation
Increased Complexity	Requires manual control over cursor lifecycle, increasing code verbosity and management efforts.
Resource Management Overhead	Consumes more system resources, especially if not closed properly.
Risk of Cursor Leaks	Forgetting to close cursors can lead to resource leaks, causing performance issues over time.
Lower Performance for Simple Operations	Less efficient than implicit cursors for simple, one-time operations.
Requires More Error Handling	Needs extra code for handling exceptions like NO_DATA_FOUND, increasing error management overhead.
Limited Scalability	May strain system resources when used heavily in high-load environments, reducing scalability.
Harder to Read and Maintain	Adds complexity to the code, making it less readable and harder to maintain, especially in large applications.

When to Use Explicit Cursors

Despite these disadvantages, explicit cursors are beneficial when:

- You need precise control over the data retrieval process (e.g., complex multi-row processing).
- You want to fetch and process data row-by-row, where each row requires specific handling.
- There's a need to work with the same set of data multiple times without re-executing the query.

In summary, while explicit cursors offer more control, they should be used judiciously to avoid the drawbacks outlined above, particularly in simple queries or high-performance environments.

Oracle – PLSQL Database Triggers

1. In Oracle PL/SQL, a **trigger** is a named block of PL/SQL code that is **automatically executed (or "fired")** in response to specific events on a particular table, view, schema, or database. **Triggers can enforce business rules, maintain audit trails, or perform automated actions.**
2. A **Trigger** is a PL/SQL block that **executes implicitly** in response to a specific database event (e.g., INSERT, UPDATE, DELETE) on a table or view. It is planned for a particular **event** (such as data changes) and **timing** (e.g., BEFORE or AFTER the event).

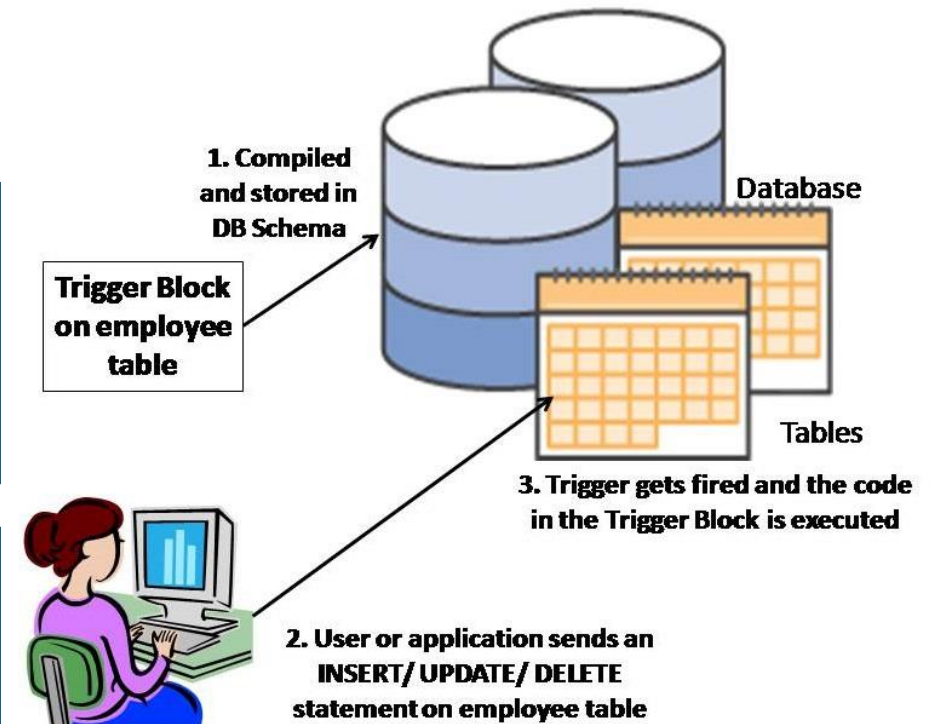


Characteristics:

- **Executed Implicitly:** Trigger fires automatically when the specified event occurs.
- **Event-Driven:** Associated with DML operations (e.g., INSERT, UPDATE, DELETE).
- **Centralized Actions:** Performs centralized and related actions to enforce data integrity, auditing, or automation.
- **Complexity:** Excessive use of triggers can lead to complex interdependencies between database objects.

Real-world Example of Trigger Usage:

- **Auditing:** Automatically log details of changes (e.g., who modified data, when, and what changed) into a separate audit table.
- **Data Validation:** Ensure specific business rules (e.g., salary cannot exceed a certain threshold) are enforced before inserting or updating a row.
- **Cascading Updates:** Update related tables to maintain data integrity when changes are made.



Source Image: <https://csveda.com/pl-sql-trigger-basics-uses/>

Triggers are a powerful feature in relational databases like Oracle, MySQL, or PostgreSQL, allowing automation and ensuring consistency within the database. However, overuse can lead to performance overhead, so they should be employed carefully.



PL/SQL Triggers – Advantages

Automation: Automates complex calculations and validations.

Data Integrity: Maintains consistency between tables (e.g., referential integrity).

Logging and Auditing: Tracks changes for insights into data usage patterns.

Real-time Synchronization: Ensures data consistency across multiple tables.

Security Enforcement: Prevents invalid transactions and restricts certain actions.

PROS

Syntax:

```
CREATE [ OR REPLACE ] TRIGGER <trigger_name>
```

```
[ BEFORE | AFTER | INSTEAD OF ]
```

Trigger Timing

```
[ INSERT | UPDATE | DELETE.....]
```

Event

```
ON <name of underlying object>
```

```
[ FOR EACH ROW ]
```

Row Level

```
[ WHEN <condition for trigger to get execute> ]
```

Conditional Clause

```
DECLARE
```

```
<Declaration part>
```

```
BEGIN
```

```
<Execution part>
```

```
EXCEPTION
```

```
<Exception handling part>
```

```
END;
```

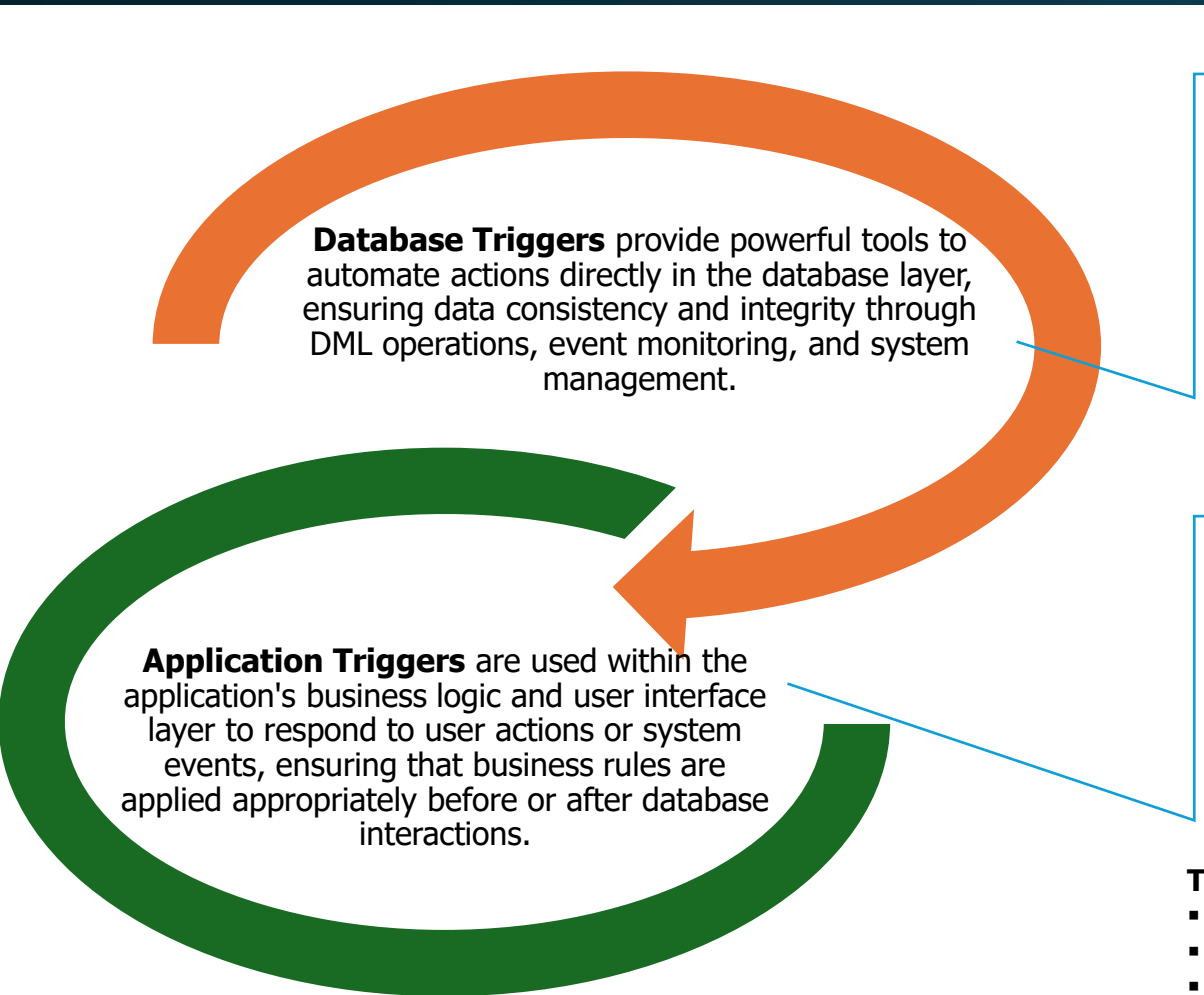
Source Image: <https://www.guru99.com/triggers-pl-sql.html>

PL/SQL Triggers – Disadvantages



Disadvantage	Description	Impact
Performance Overhead	Triggers add processing time to INSERT, UPDATE, or DELETE operations.	Can slow down database performance, especially for high-traffic tables or large datasets.
Debugging and Maintenance	Triggers execute implicitly, making them hard to debug and trace.	Difficult for developers to track issues or understand the flow of data modifications.
Hidden Logic	Business logic in triggers is not immediately visible in application code.	Causes confusion and reduces transparency, leading to unexpected behavior.
Increased Complexity	Triggers can cause unintended cascading effects (e.g., one trigger firing another).	Creates complex dependencies, making the database harder to manage.
Limited Use Cases	Triggers are bound to database events and cannot handle all types of application logic.	Requires additional methods (e.g., stored procedures or application logic) for more complex scenarios.
Recursive Execution	Poorly designed triggers may invoke themselves or other triggers recursively.	Can lead to infinite loops, database crashes, or data corruption.
Portability Issues	Trigger syntax and behavior vary across database platforms.	Challenges arise when migrating to a different database system.
Difficulty in Testing	Triggers execute in the background, making them hard to test independently.	Ensuring expected behavior in all scenarios requires extensive testing.
Over-reliance on Triggers	Developers may use triggers for tasks better handled at the application layer.	Blurs the separation between application and database logic, complicating system maintenance.

Triggers Classification: Application & Database Trigger



Database Triggers provide powerful tools to automate actions directly in the database layer, ensuring data consistency and integrity through DML operations, event monitoring, and system management.

```
CREATE OR REPLACE TRIGGER check_salary
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    IF :NEW.salary < 1000 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Salary must be above 1000');
    END IF;
END;
```

```
// Java pseudo code example to demonstrate application trigger
public void onSalaryInsert(Employee emp) {
    if (emp.getSalary() < 1000) {
        throw new IllegalArgumentException("Salary must be greater than or
        equal to 1000");
    }
    // Code to insert employee to the database
}
```

Application Triggers are used within the application's business logic and user interface layer to respond to user actions or system events, ensuring that business rules are applied appropriately before or after database interactions.

Types:

- **UI Event Handlers:** Trigger actions when a user interacts with the interface (e.g., button clicks).
- **Business Logic Triggers:** Trigger specific actions or calculations based on certain conditions.
- **Database Interactions:** Trigger actions after database operations (e.g., after a record is updated).
- **Use Cases:** Business rule enforcement, user interaction handling, database operation responses

Trigger Syntax, Behavior of Database Triggers

```
-- Creates or replaces a trigger to execute logic before/after specific table events.
CREATE [OR REPLACE] TRIGGER trigger_name

-- Determines trigger timing relative to the event (BEFORE, AFTER, or INSTEAD OF).
{BEFORE | AFTER | INSTEAD OF}

-- Specifies the triggering events (INSERT, UPDATE, DELETE).
{INSERT [OR] | UPDATE [OR] | DELETE}

-- Monitors specific columns for UPDATE events (optional).
[OF col_name]

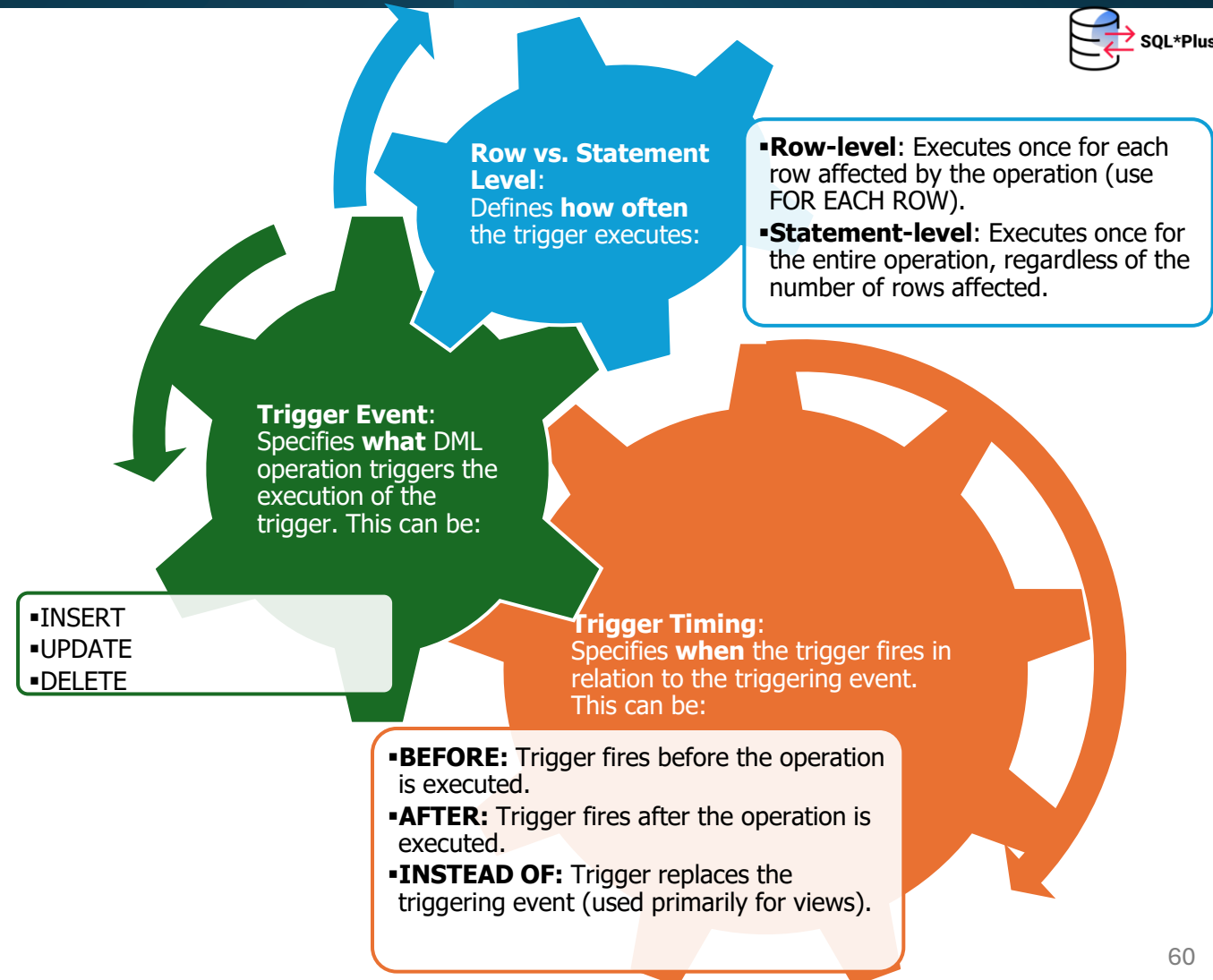
-- Defines the table or view the trigger applies to.
ON table_name

-- References old and new row values for processing (optional).
[REFERENCING OLD AS o NEW AS n]

-- Executes the trigger for each affected row (optional).
[FOR EACH ROW]

-- Executes the trigger conditionally if the specified condition is true.
WHEN (condition)

BEGIN
-- Define the logic executed when the trigger fires.
NULL; -- Replace with actual trigger logic.
EXCEPTION
-- Define logic for handling errors during trigger execution.
NULL; -- Replace with actual error handling logic
END;
/
```



Where,

CREATE [OR REPLACE] TRIGGER trigger_name – Creates or replaces an existing trigger with the *trigger_name*.

{BEFORE | AFTER | INSTEAD OF} – This specifies when the trigger will be executed. The INSTEAD OF clause is used for creating trigger on a view.

{INSERT [OR] | UPDATE [OR] | DELETE} – This specifies the DML operation.

[OF col_name] – This specifies the column name that will be updated.

[ON table_name] – This specifies the name of the table associated with the trigger.

[REFERENCING OLD AS o NEW AS n] – This allows you to refer new and old values for various DML statements, such as INSERT, UPDATE, and DELETE.

[FOR EACH ROW] – This specifies a row-level trigger, i.e., the trigger will be executed for each row being affected. Otherwise the trigger will execute just once when the SQL statement is executed, which is called a table level trigger.

WHEN (condition) – This provides a condition for rows for which the trigger would fire. This clause is valid only for row-level triggers.

When to Use Triggers

1. DML Triggers:

Use **DML Triggers** to automate actions in response to changes in data. These are ideal for:

- Logging changes to tables (e.g., tracking updates or deletions).
- Enforcing business rules and integrity constraints automatically when INSERT, UPDATE, or DELETE operations occur.

2. INSTEAD OF Triggers:

Use **INSTEAD OF Triggers** to enable INSERT, UPDATE, or DELETE operations on views that are otherwise non-updatable. These triggers allow you to define custom logic for handling DML operations on such views.

3. DDL Triggers:

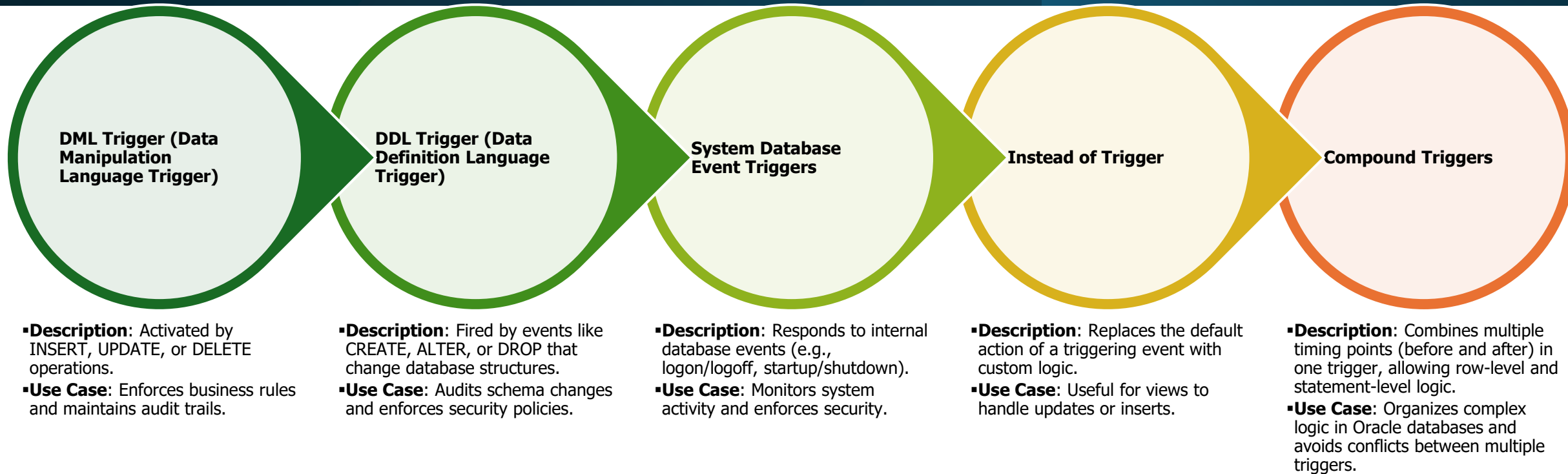
Use **DDL Triggers** to monitor or control schema-level changes. These triggers are helpful for:

- Auditing schema modifications (e.g., DROP, ALTER, or CREATE operations).
- Restricting unauthorized schema changes at the database or schema level.

Caution with Triggers

- Avoid excessive use of triggers to prevent **complex interdependencies**.
- Excessive use can degrade performance and make debugging difficult.
- Disable or drop triggers temporarily when troubleshooting or performing bulk operations.

Types of Database Triggers



Summary

These triggers help automate actions, enforce rules, and maintain data integrity in database operations.



Table Auditing using DML Triggers in Oracle PL/SQL

Auditing Using Triggers vs Auditing Vault: A Detailed Comparison

Both **triggers** and **auditing vaults** are tools used to track and monitor database activities, but they serve different purposes and operate at distinct levels. Here's a detailed breakdown:

Feature	Auditing Using Triggers	Auditing Vault
Scope	Table/View-specific DML operations	Comprehensive (DML, DDL, and system events)
Performance Impact	Can affect application performance	Minimal impact with optimized configurations
Ease of Implementation	Requires coding and testing for each trigger	Requires configuration but no coding
Customizability	Fully customizable	Limited to predefined templates and policies
Tamper Resistance	Can be bypassed by disabling triggers	Logs are secured and tamper-proof
Cost	No additional cost beyond database licensing	Often part of enterprise solutions with extra fees
Regulatory Compliance	Limited (manual efforts required)	Designed for compliance with built-in tools

I. Problem Statement - Scenario in a Banking Context DDL Trigger

In a bank's database system, schema-level changes such as creating or altering tables for customer records, transaction logs, or account details must be strictly monitored for compliance and security purposes. You are tasked with implementing a mechanism to track and log all **DDL (Data Definition Language)** operations like CREATE, ALTER, and DROP. This ensures that unauthorized or accidental modifications to critical database structures are detected and auditable.

Objectives:

1. Create an Audit Table

Design a table that logs schema-level events, capturing:

- Event type (CREATE, ALTER, DROP).
- Object name (e.g., customer_accounts, transaction_history).
- User performing the operation (e.g., DB_ADMIN).
- Timestamp of the change.

2. Write a DDL Trigger

Implement a trigger that:

- Fires automatically after any CREATE, ALTER, or DROP operation on the database.
- Inserts event details into the audit table.

3. Test the Solution

Perform test operations relevant to a bank:

- Create a new table for storing loan applications (CREATE TABLE loan_applications...).
- Alter an existing table to add a column for interest rates (ALTER TABLE...).
- Drop an obsolete table for temporary data storage (DROP TABLE temp_data...).

4. Validate the Implementation

Query the audit log to ensure the trigger:

- Captures the operation type and object name (e.g., CREATE TABLE loan_applications).
- Logs the user performing the operation and the time.



Example Use Case:

- A bank's database administrator creates a new table for storing customer feedback.
- The trigger logs this CREATE operation into the audit table, noting the table name, the admin's username, and the timestamp.
- If unauthorized schema changes occur, the audit log provides a detailed trail for investigation and accountability.

Solution DDL Trigger: Banque Scenario

Step 1: Create the Audit Table

The audit table will store details of all CREATE, ALTER, and DROP operations.

```
CREATE TABLE audit_ddl_log (  
  id NUMBER GENERATED BY DEFAULT AS IDENTITY,  
  event_type VARCHAR2(50),  
  object_name VARCHAR2(100),  
  operation_user VARCHAR2(50),  
  timestamp DATE  
);
```

Step 2: Create the DDL Trigger

This trigger will log relevant details whenever a DDL operation occurs.

```
CREATE OR REPLACE TRIGGER log_ddl_operations  
AFTER CREATE OR ALTER OR DROP ON DATABASE  
BEGIN  
  INSERT INTO audit_ddl_log (event_type, object_name, operation_user, timestamp)  
  VALUES (  
    ORA_SYSEVENT,           -- Captures the type of event (e.g., CREATE, ALTER, DROP)  
    ORA_DICT_OBJ_NAME,      -- Captures the affected object (e.g., table name)  
    SYS_CONTEXT('USERENV', 'SESSION_USER'), -- Captures the user performing the  
    operation  
    SYSDATE                 -- Captures the timestamp of the operation  
  );  
END;  
/
```

Step 3. Testing

Test Case 1: Creating a Table

Execute the following command:

```
CREATE TABLE customer_feedback (  
  feedback_id NUMBER,  
  feedback_text VARCHAR2(255)  
);
```

Validate: Query the audit log:

```
SELECT * FROM audit_ddl_log;
```

Test Case 2: Altering a Table

Execute the following command:

```
ALTER TABLE customer_feedback ADD feedback_date DATE;
```

Validate: Query the audit log:

```
ALTER TABLE customer_feedback ADD feedback_date DATE;
```

Test Case 3: Dropping a Table

Execute the following command:

```
DROP TABLE customer_feedback;
```

Validate: Query the audit log:

```
SELECT * FROM audit_ddl_log;
```

Expected Output

ID	EVENT_TYPE	OBJECT_NAME	OPERATION_USER	TIMESTAMP
1	CREATE	CUSTOMER_FEEDBACK	DB_ADMIN	2024-11-20 14:00:00
2	ALTER	CUSTOMER_FEEDBACK	DB_ADMIN	2024-11-20 14:10:00
3	DROP	CUSTOMER_FEEDBACK	DB_ADMIN	2024-11-20 14:20:00

Solution DDL Trigger: Banque Scenario

Summary of the Solution

1. **Audit Table:** Stores details about schema changes for accountability.
2. **DDL Trigger:** Automatically logs all CREATE, ALTER, and DROP operations.
3. **Testing:** Verified the trigger by performing sample DDL operations and checking the audit log.

This implementation provides a robust mechanism to monitor and log critical schema changes in a bank's database, ensuring security and compliance.

Note:

- **You do not manually call a trigger.** Instead, it gets triggered automatically when the defined DML operation (INSERT, UPDATE, DELETE) happens on the specified table (in this case, EMPLOYEES).
- **To test it,** simply perform the corresponding DML operation (insert, update, or delete) on the EMPLOYEES table.
- **To see the output of the trigger,** ensure DBMS_OUTPUT is enabled and perform the operations.



II. Creating a DML Trigger for Auditing Allowances Table Operations

You are tasked with implementing a mechanism to track and log all DML (Data Manipulation Language) operations (INSERT, UPDATE, DELETE) on key tables in a banking system database (such as ALLOWANCES, ATTENDANCE, COUNTRIES, DEPARTMENTS, EMPLOYEES, ROLES). This functionality is required to ensure proper auditing and accountability of schema-level changes.

Task:

- 1. Create an audit table** to log the details of the DML operations. The audit table should capture:
 - Type of operation (INSERT, UPDATE, DELETE)
 - The name of the affected table
 - The user who performed the operation
 - The timestamp of when the operation occurred
 - The old and new data for UPDATE operations
- 2. Write a DML trigger** for the ALLOWANCES table (you may also create similar triggers for other tables) that:
 - Fires automatically after any INSERT, UPDATE, or DELETE operation on the ALLOWANCES table.
 - Inserts relevant details into the audit table (as specified above).
- 3. Test the functionality** by performing the following operations on the ALLOWANCES table:
 - Insert a new allowance record.
 - Update an existing allowance record.
 - Delete an allowance record.
4. After performing the operations, query the audit table to verify that the changes have been properly logged.



Requirements:

- The audit table should include:
 - operation_type (INSERT, UPDATE, DELETE)
 - object_name (the affected table)
 - operation_user (user who performed the operation)
 - timestamp (time of operation)
 - old_data (before the operation) and new_data (after the operation), as applicable
- Your trigger should capture the following:
 - **For INSERT:** Log the new data inserted.
 - **For UPDATE:** Log both the old and new data.
 - **For DELETE:** Log the old data before the row is deleted.

Evaluation:

- Does the trigger correctly capture and log the DML operations?
- Is the audit table populated with the correct details after each DML operation?
- Are all the required fields logged properly (operation type, affected table, user, timestamp, old/new data)?

Solution - DML Trigger - Auditing Allowances Operations (1/2)

`SYS_CONTEXT('USERENV', 'SESSION_USER')` is an Oracle SQL function used to retrieve information about the current session, specifically the session user (i.e., the user currently connected to the database).

```
/*
Step 1: Create the Audit Table
First, let's create an audit table that logs the changes made to any of these tables.
*/
CREATE TABLE audit_dml_log (
  id NUMBER GENERATED BY DEFAULT AS IDENTITY, -- Unique identifier
  operation_type VARCHAR2(50),                -- Type of operation (INSERT, UPDATE, DELETE)
  object_name VARCHAR2(100),                  -- Name of the affected table
  operation_user VARCHAR2(50),                -- User who performed the operation
  timestamp DATE,                            -- Date and time of the operation
  old_data VARCHAR2(4000),                    -- Old data (for UPDATE and DELETE)
  new_data VARCHAR2(4000)                     -- New data (for INSERT and UPDATE)
);

/*
Step 2: Create the DML Trigger
Now, let's create a DML Trigger for each table (e.g., ALLOWANCES, ATTENDANCE, COUNTRIES, DEPARTMENTS,
EMPLOYEES, ROLES). This trigger will capture the changes made (insertion, update, deletion)
to any of the tables and log those changes in the audit_dml_log table.
*/

CREATE OR REPLACE TRIGGER log_dml_operations
AFTER INSERT OR UPDATE OR DELETE ON ALLOWANCES -- You can change this table name dynamically if needed
FOR EACH ROW
BEGIN
  IF INSERTING THEN
    INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, new_data)
    VALUES ('INSERT', 'ALLOWANCES', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE,
:NEW.ALLOWANCE_AMOUNT);
  ELSIF UPDATING THEN
    INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, old_data, new_data)
    VALUES ('UPDATE', 'ALLOWANCES', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE,
:OLD.ALLOWANCE_AMOUNT, :NEW.ALLOWANCE_AMOUNT);
  ELSIF DELETING THEN
    INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, old_data)
    VALUES ('DELETE', 'ALLOWANCES', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE,
:OLD.ALLOWANCE_AMOUNT);
  END IF;
END;
/
```

Solution - DML Trigger - Auditing Allowances Operations (2/3)

```
/*
```

Step 3: Repeat for Other Tables

You can repeat the process for other tables (ATTENDANCE, COUNTRIES, DEPARTMENTS, EMPLOYEES, ROLES) by changing the AFTER INSERT OR UPDATE OR DELETE ON part of the trigger, and modifying the field names in the :NEW and :OLD references according to the specific columns in those tables.

For example, for the ATTENDANCE table, the trigger would look like this:

```
*/
```

```
CREATE OR REPLACE TRIGGER log_dml_operations_attendance
```

```
AFTER INSERT OR UPDATE OR DELETE ON ATTENDANCE
```

```
FOR EACH ROW
```

```
BEGIN
```

```
  IF INSERTING THEN
```

```
    INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, new_data)
```

```
    VALUES ('INSERT', 'ATTENDANCE', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE, :NEW.STATUS);
```

```
  ELSIF UPDATING THEN
```

```
    INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, old_data, new_data)
```

```
    VALUES ('UPDATE', 'ATTENDANCE', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE, :OLD.STATUS,
```

```
:NEW.STATUS);
```

```
  ELSIF DELETING THEN
```

```
    INSERT INTO audit_dml_log (operation_type, object_name, operation_user, timestamp, old_data)
```

```
    VALUES ('DELETE', 'ATTENDANCE', SYS_CONTEXT('USERENV', 'SESSION_USER'), SYSDATE, :OLD.STATUS);
```

```
  END IF;
```

```
END;
```

```
/
```

Test - DML Trigger - Auditing Allowances Operations (3/3)

```
/*  
Step 4: Test the Trigger  
Let's perform some operations on the ALLOWANCES table to verify that the trigger is working as expected.
```

```
Test Case 1: Inserting a Row  
*/
```

```
INSERT INTO ALLOWANCES (ALLOWANCE_ID, ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE,  
COMMENTS)  
VALUES ( 12, 2, 5000, TO_DATE('2024-11-01', 'YYYY-MM-DD'), 'Y', 'Performance-based allowance');
```

```
/*  
Check the audit log:  
*/  
SELECT * FROM audit_dml_log;
```

```
/*  
Test Case 2: Updating a Row  
*/  
UPDATE ALLOWANCES  
SET ALLOWANCE_AMOUNT = 5500  
WHERE ALLOWANCE_ID = 12;  
/*  
Check the audit log:  
*/
```

```
SELECT * FROM audit_dml_log;
```

```
/*  
Test Case 3: Deleting a Row  
*/
```

```
DELETE FROM ALLOWANCES WHERE ALLOWANCE_ID = 12;
```

```
/*  
Check the audit log:  
*/  
SELECT * FROM audit_dml_log;
```

System Event Triggers

A **system event trigger** is a special type of database trigger that executes automatically in response to specific events occurring at the **system level** (e.g., user logins, database startup/shutdown, schema-level changes). These triggers are not tied to a specific table but are linked to system or database events.

Common Use Cases for System Event Triggers:

- **LOGON**: Triggered when a user logs into the database.
- **LOGOFF**: Triggered when a user logs off from the database.
- **STARTUP**: Triggered when the database is started.
- **SHUTDOWN**: Triggered when the database is shut down.
- **AFTER SERVERERROR**: Triggered after a server error occurs.
- **AFTER DDL**: Triggered after a Data Definition Language (DDL) statement (e.g., CREATE, ALTER, DROP) is executed.
- **BEFORE DDL**: Triggered before a DDL statement is executed.

```
-- Create or replace the trigger
CREATE OR REPLACE TRIGGER trigger_name

-- Specify whether the trigger fires BEFORE or AFTER the event
AFTER | BEFORE

-- Define the system event to monitor (e.g., STARTUP, SHUTDOWN, LOGON, LOGOFF, DDL, SERVERERROR)
[STARTUP | SHUTDOWN | LOGON | LOGOFF | DDL | SERVERERROR]

-- Specify the scope of the trigger (DATABASE for all users or SCHEMA for the current schema)
ON [DATABASE | SCHEMA]

BEGIN
    -- Add your PL/SQL code here to define what the trigger should do
END;
/
```

Example 1: Logging User Logins

```
/*  
Check the USER_LOG Table:  
Log in to the database as the user who created the  
trigger and verify the records.  
Query the USER_LOG table:  
*/
```

```
SELECT * FROM USER_LOG;
```

This trigger captures login events and logs them into a custom USER_LOG table.

Create Audit Table for Login Logs

```
-- Create a table to log user activity  
CREATE TABLE USER_LOG (  
    LOG_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, -- Unique identifier for each log entry  
    USERNAME VARCHAR2(50), -- Stores the username of the logged-in user  
    SESSION_ID NUMBER, -- Tracks the session ID of the user  
    LOGON_TIME TIMESTAMP DEFAULT CURRENT_TIMESTAMP -- Records the logon time (defaults to the current  
time)  
);
```

Create a trigger

```
-- Create or replace a trigger named 'trg_logon_audit'  
CREATE OR REPLACE TRIGGER trg_logon_audit  
  
-- Specify that the trigger fires AFTER a LOGON event on the entire DATABASE  
AFTER LOGON ON DATABASE  
BEGIN  
    -- Insert a record into the USER_LOG table for every user logon  
    INSERT INTO USER_LOG (USERNAME, SESSION_ID, LOGON_TIME)  
  
    -- USER: Captures the username of the logged-in user  
    -- SYS_CONTEXT('USERENV', 'SESSIONID'): Retrieves the current session ID  
    -- SYSTIMESTAMP: Captures the current timestamp for the logon  
    VALUES (USER, SYS_CONTEXT('USERENV', 'SESSIONID'), SYSTIMESTAMP);  
END trg_logon_audit;  
/
```


Example 2: Logging DDL Changes

/*
Step 1: Ensure the DDL_LOG Table Exists
Before testing, make sure the DDL_LOG table is
created and ready to store audit records:
*/
SELECT * FROM DDL_LOG;

This trigger tracks all Data Definition Language (DDL) operations (e.g., CREATE, ALTER, DROP) performed in the schema.

Create Audit Table for DDL Logs:

```
CREATE TABLE DDL_LOG (  
  LOG_ID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  USERNAME VARCHAR2(50),  
  SCHEMA_NAME VARCHAR2(50),  
  EVENT_TYPE VARCHAR2(50),  
  OBJECT_NAME VARCHAR2(100),  
  TIMESTAMP TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Create a trigger

```
-- Create or replace a trigger named 'trg_ddl_audit'  
CREATE OR REPLACE TRIGGER trg_ddl_audit  
  
-- Specify that the trigger fires AFTER any DDL (Data Definition Language) operation on the current schema  
AFTER DDL ON SCHEMA  
BEGIN  
  -- Insert a record into the DDL_LOG table for every DDL operation  
  INSERT INTO DDL_LOG (  
    USERNAME,      -- The user performing the DDL operation  
    SCHEMA_NAME,   -- The schema where the DDL operation is being executed  
    EVENT_TYPE,    -- The type of DDL event (e.g., CREATE, ALTER, DROP)  
    OBJECT_NAME,   -- The name of the object affected by the DDL event (e.g., table, view, etc.)  
    TIMESTAMP      -- The timestamp when the DDL operation occurred  
  )  
  VALUES (  
    USER,          -- Built-in function returning the username  
    SYS_CONTEXT('USERENV', 'CURRENT_SCHEMA'), -- Retrieves the current schema name  
    ORA_SYSEVENT,  -- Built-in variable for the type of DDL event (e.g., CREATE, ALTER, DROP)  
    ORA_DICT_OBJ_NAME, -- Built-in variable for the name of the object involved in the DDL  
    operation  
    SYSTIMESTAMP   -- Captures the current date and time  
  );  
END trg_ddl_audit;  
/
```

Example 3: Logging Shutdown Events

-- Verify the Trigger Execution

```
SELECT * FROM shutdown_log ORDER BY event_time  
DESC;
```

-- Step 1: Create a table to log shutdown events

```
CREATE TABLE shutdown_log (  
  log_id NUMBER GENERATED BY DEFAULT AS IDENTITY,  
  event_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  event_message VARCHAR2(4000)  
);
```

-- Step 2: Create a system-level trigger

```
CREATE OR REPLACE TRIGGER trg_on_shutdown  
AFTER SERVERERROR ON DATABASE  
BEGIN
```

-- Check if the error indicates a shutdown

```
IF (ora_is_servererror(1033)) THEN
```

```
  INSERT INTO shutdown_log (event_message)
```

```
  VALUES ('Database shutdown initiated.');
```

```
END IF;
```

```
EXCEPTION
```

```
  WHEN OTHERS THEN
```

-- Handle any exceptions to ensure the database trigger doesn't fail

```
  NULL;
```

```
END;
```

```
/
```

-- Explanation:

-- ORA-01033 is a common error code indicating "Oracle initialization or shutdown in progress."

-- This trigger will log a message whenever a shutdown event is detected by the database.

```
/*
```

Simulate Database Shutdown:

You can initiate a database shutdown using the following SQL command (requires DBA privileges):

```
*/
```

```
SHUTDOWN IMMEDIATE;
```

```
/*
```

Check the Logs:

```
*/
```

```
SELECT * FROM shutdown_log;
```

Trigger Event: Enforcing HR Transaction Restrictions During Non-Working Hours

Scenario:

The **Human Resources Department** of a large organization manages allowances for its employees. As part of the organizational policy, it is required that any changes to the **ALLOWANCES** table—whether it be inserting new allowances, updating existing records, or deleting them—should only be allowed during official working hours (from **6:00 AM to 6:00 PM**). This policy is designed to ensure that only authorized personnel, during working hours, can make changes to employee allowances. If an attempt is made to insert, update, or delete records outside of these hours (e.g., during the night or weekends), the system should automatically block such attempts and raise an appropriate error message to inform the user of the restriction.

Task:

As a database developer, you are tasked with implementing a PL/SQL trigger that enforces this policy on the **ALLOWANCES** table. The trigger should:

1. Fire **before any INSERT, UPDATE, or DELETE** operation on the **ALLOWANCES** table.
2. Check the system's current time to ensure it falls within the allowed timeframe of **6:00 AM to 6:00 PM**
3. If the operation is attempted outside these hours, raise an exception with the following message:
"Operations on the ALLOWANCES table are only allowed between 6:00 AM and 6:00 PM."

Deliverable:

- Write a PL/SQL trigger that implements the described functionality.



NEW is a reference to the new values being inserted or updated in the table. Specifically, **:NEW** refers to the record being affected by the INSERT or UPDATE operation in the **ALLOWANCES** table.

Solution: Enforcing HR Transaction Restriction

Great things never come from comfort zones—dare to code.

```
CREATE OR REPLACE TRIGGER restrict_allowance_insert_hours
BEFORE INSERT OR UPDATE OR DELETE ON ALLOWANCES
FOR EACH ROW
DECLARE
    -- Declare an exception to handle invalid transaction time
    invalid_time EXCEPTION;
BEGIN
    -- Check the current time in HH24MI format (24-hour clock without colon)
    IF TO_CHAR(SYSDATE, 'HH24MI') NOT BETWEEN '0600' AND '1800' THEN
        -- Raise the exception if the current time is outside working hours
        RAISE invalid_time;
    END IF;
    -- Optional: Additional validation for the data being modified
    IF :NEW.ALLOWANCE_AMOUNT < 0 THEN
        -- If allowance amount is invalid (negative), raise an exception
        RAISE_APPLICATION_ERROR(-20003, 'Allowance amount cannot be negative.');
```



```
    END IF;
    IF :NEW.EFFECTIVE_DATE < SYSDATE THEN
        -- If the effective date is in the past, raise an exception
        RAISE_APPLICATION_ERROR(-20004, 'Effective date cannot be in the past.');
```



```
    END IF;

    IF :NEW.IS_APPLICABLE IS NULL THEN
        -- If IS_APPLICABLE is not provided, raise an exception
        RAISE_APPLICATION_ERROR(-20005, 'Applicability status (IS_APPLICABLE) is required.');
```



```
    END IF;

    -- Optional: Check that COMMENTS field is not empty or null
    IF :NEW.COMMENTS IS NULL OR TRIM(:NEW.COMMENTS) IS NULL THEN
        -- If no comments are provided, raise an exception
        RAISE_APPLICATION_ERROR(-20006, 'Comments are required.');
```



```
    END IF;
EXCEPTION
    WHEN invalid_time THEN
        -- Handle invalid time exception with custom message
        RAISE_APPLICATION_ERROR(-20002, 'Operations on the ALLOWANCES table are only allowed between 6:00 AM and 6:00 PM.');
```



```
    WHEN OTHERS THEN
        -- Generic exception handler for unexpected errors
        RAISE_APPLICATION_ERROR(-20001, 'An unexpected error occurred: ' || SQLERRM);
END;
/
```

Test: Enforcing HR Transaction Restriction Trigger



```
/*
1. Valid Transaction (Within Working Hours):
*/
-- Example: Insert at 10:00 AM with valid data
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (1, 500, SYSDATE + 1, 'Y', 'Annual Bonus');
-- Success: Operation allowed.

/*
2. Invalid Transaction (Outside Working Hours):
*/
-- Example: Insert at 8:00 PM (outside working hours)
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (2, 600, SYSDATE + 1, 'Y', 'Holiday Allowance');
-- Error: "Operations on the ALLOWANCES table are only allowed between 6:00 AM and 6:00 PM."

/*
3. Invalid Transaction (Negative Allowance Amount):
*/
-- Example: Insert with a negative allowance amount
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (3, -200, SYSDATE + 1, 'Y', 'Error Test');
-- Error: "Allowance amount cannot be negative."

/*
4. Invalid Transaction (Effective Date in the Past):
*/
-- Example: Insert with an effective date in the past
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (4, 300, SYSDATE - 1, 'Y', 'Retroactive Allowance');
-- Error: "Effective date cannot be in the past."

/*
5. Invalid Transaction (Null Comments):
*/
-- Example: Insert with no comments provided
INSERT INTO ALLOWANCES (ROLE_ID, ALLOWANCE_AMOUNT, EFFECTIVE_DATE, IS_APPLICABLE, COMMENTS)
VALUES (5, 400, SYSDATE + 1, 'Y', NULL);
-- Error: "Comments are required."
```

Trigger Event: Restricting Bank Transactions Outside Working Hours

Scenario:

The Bank of Kigali has implemented a policy to restrict tellers from recording transactions outside official working hours to ensure operational compliance and security.

Requirements:

- Transactions can only be recorded **Monday to Friday**, between **9:00 AM and 6:00 PM**.
- Any attempt to record a transaction outside these hours or on weekends must be **blocked automatically** by the database system.

Task for the Database Developer:

As a database developer, your responsibility is to:

1. Create a PL/SQL Trigger:

- The trigger should fire **BEFORE INSERT** or **BEFORE UPDATE** on the transactions table.

2. Validation Logic:

- The trigger must check the current system date and time (SYSDATE) to validate the transaction timing.

3. Exception Handling:

- If a teller attempts to record a transaction outside the allowed timeframe, the trigger should:
 - **Raise an exception** with the message:
"*Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday.*"

This ensures that all transactions adhere to the defined business rules and operational policies.

Now I can confidently audit a bank database!
Using the concept of logging shutdown events,
I can track critical activities, ensure
accountability, and monitor unauthorized
actions. This practical approach to auditing
demonstrates how to enhance security and
compliance while building robust insights for
database management.



Trigger Event: Restricting Bank Transactions Outside Working Hours – Solution

```
-- Step 1: Create a sample transactions table (if it doesn't already exist)
CREATE TABLE transactions (
    transaction_id NUMBER PRIMARY KEY,
    teller_id NUMBER,
    transaction_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    amount NUMBER
);

-- Step 2: Create the trigger to enforce the policy
CREATE OR REPLACE TRIGGER trg_validate_transaction_time
BEFORE INSERT OR UPDATE ON transactions
FOR EACH ROW
BEGIN
    -- Get the current day of the week and time
    IF TO_CHAR(SYSDATE, 'DY', 'NLS_DATE_LANGUAGE=ENGLISH') IN ('SAT', 'SUN') THEN
        -- Raise an exception for weekends
        RAISE_APPLICATION_ERROR(-20001,
            'Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday.');
```

Invalid Transaction (Weekend):

```
    */
    -- Example of a transaction attempted on a Saturday
    INSERT INTO transactions (transaction_id, teller_id, amount)
    VALUES (2, 102, 1000);
    -- Error: "Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday."

    /*
    Invalid Transaction (Outside Working Hours):
    */
    -- Example of a transaction attempted at 8:00 PM on a Monday
    INSERT INTO transactions (transaction_id, teller_id, amount)
    VALUES (3, 103, 1500);
    -- Error: "Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday."

```

```
    */
    -- Example of a valid transaction (Monday at 10:00 AM)
    INSERT INTO transactions (transaction_id, teller_id, amount)
    VALUES (1, 101, 500);
    -- Success: Transaction is recorded.

    /*
    Invalid Transaction (Weekend):
    */
    -- Example of a transaction attempted on a Saturday
    INSERT INTO transactions (transaction_id, teller_id, amount)
    VALUES (2, 102, 1000);
    -- Error: "Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday."

    /*
    Invalid Transaction (Outside Working Hours):
    */
    -- Example of a transaction attempted at 8:00 PM on a Monday
    INSERT INTO transactions (transaction_id, teller_id, amount)
    VALUES (3, 103, 1500);
    -- Error: "Transactions can only be recorded between 9:00 AM and 6:00 PM, Monday to Friday."

```



Trigger Event: Before Update Trigger to Enforce Salary Protection in Employees Table

Question:

Suppose you are managing an employees table where each employee has a salary that should not be decreased. Write a trigger named prevent_salary_reduction that prevents any update to the salary column if the new salary is lower than the existing salary.

The trigger should meet the following requirements:

1. It should be a **BEFORE UPDATE** trigger, so it checks the salary change before the update is applied.
2. It should raise an error with the message **"Salary reduction is not allowed."** if an attempt is made to reduce an employee's salary.
3. Use RAISE_APPLICATION_ERROR with a custom error code to handle this restriction.

Provide the full trigger code and explain each part of the code in comments.



```
-- Trigger to prevent salary reduction in the 'employees' table
CREATE OR REPLACE TRIGGER prevent_salary_reduction
-- The trigger will fire before any update on the salary column in the employees table
BEFORE UPDATE OF salary ON employees
FOR EACH ROW -- Specifies that the trigger will execute once for each row being updated
BEGIN
    -- Check if the new salary is less than the old salary
    IF :NEW.salary < :OLD.salary THEN
        -- Raise an application error if the new salary is less than the old salary
        RAISE_APPLICATION_ERROR(-20001, 'Salary reduction is not allowed.');
```

Creating a Trigger to Log Employee Salary Changes and Operations

BEFORE INSERT OR UPDATE OR DELETE



Suppose you are tasked with creating a trigger that logs salary changes for employees, but only for those whose `ROLE_ID` is not equal to 1 (assuming role 1 is the Manager). The trigger should work for `INSERT`, `UPDATE`, or `DELETE` operations on the `EMPLOYEES` table.

Write a trigger named `log_salary_changes` that fulfills the following requirements:

1. The trigger should be fired **before** an `INSERT`, `UPDATE`, or `DELETE` operation on the `EMPLOYEES` table.
2. It should log the following details for each type of operation:
 - For an `UPDATE` operation: The old and new salary values and the difference between them.
 - For an `INSERT` operation: The name of the new employee and their starting salary.
 - For a `DELETE` operation: The name of the deleted employee and their final salary.
3. The trigger should only log changes for employees who do not have the role of Manager (`ROLE_ID <> 1`).
4. Make use of the `DBMS_OUTPUT.PUT_LINE` procedure to display the logged information.

You should handle all operations appropriately within the trigger, using the `UPDATING`, `INSERTING`, and `DELETING` conditions to check the operation type.

Hint: Use the `:NEW` and `:OLD` references to capture the values of the columns before and after the operations.



Trigger Event: Log Employee Salary Changes and Operations – Solution



```
CREATE OR REPLACE TRIGGER log_salary_changes
BEFORE INSERT OR UPDATE OR DELETE ON EMPLOYEES
FOR EACH ROW
WHEN (NEW.ROLE_ID <> 1) -- Assuming '1' is the role ID for the Manager
DECLARE
    sal_diff NUMBER; -- Variable to hold the salary difference
BEGIN
    -- Check if the operation is an update to log salary changes
    IF UPDATING THEN
        sal_diff := :NEW.SALARY - :OLD.SALARY;
        DBMS_OUTPUT.PUT_LINE('Updating Employee: ' || :NEW.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Old salary = ' || :OLD.SALARY || ', New salary = ' || :NEW.SALARY);
        DBMS_OUTPUT.PUT_LINE('Difference: ' || sal_diff);
    ELSIF INSERTING THEN
        DBMS_OUTPUT.PUT_LINE('Inserting New Employee: ' || :NEW.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Starting Salary: ' || :NEW.SALARY);
    ELSIF DELETING THEN
        DBMS_OUTPUT.PUT_LINE('Deleting Employee: ' || :OLD.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Final Salary: ' || :OLD.SALARY);
    END IF;
END;
/

/*
Test Insert Operation
*/
INSERT INTO EMPLOYEES (EMPLOYEE_ID, FIRST_NAME, LAST_NAME, ROLE_ID, SALARY)
VALUES (1001, 'John', 'Doe', 2, 4000); -- Assuming role ID '2' is for a non-manager

/* Test Update Operation */
UPDATE EMPLOYEES
SET SALARY = 4500
WHERE EMPLOYEE_ID = 1001;
```

Hint: Use the :NEW and :OLD references to capture the values of the columns before and after the operations.

Difference Between BEFORE and INSTEAD OF Triggers

BEFORE Trigger

Executes before the DML operation takes place. The operation is still carried out, unless you explicitly raise an error or take other actions to stop it.

INSTEAD OF Trigger

Completely replaces the DML operation. It allows you to define custom logic and replace the default behavior of INSERT, UPDATE, or DELETE.

For example, if you have a view and you want to control what happens when a user inserts, updates, or deletes data, you use an INSTEAD OF trigger.

INSTEAD OF Trigger

```
/*
1. Create the View (if not already created):
*/
CREATE OR REPLACE VIEW EMPLOYEES_VIEW AS
SELECT EMPLOYEE_ID, LAST_NAME, SALARY
FROM EMPLOYEES;

/*
2. Create the INSTEAD OF Trigger:
*/
CREATE OR REPLACE TRIGGER log_salary_changes
INSTEAD OF INSERT OR UPDATE OR DELETE ON EMPLOYEES_VIEW
FOR EACH ROW
DECLARE
    sal_diff NUMBER;
BEGIN
    -- Handling INSERT operation
    IF INSERTING THEN
        DBMS_OUTPUT.PUT_LINE('Inserting New Employee: ' || :NEW.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Starting Salary: ' || :NEW.SALARY);
        -- Perform actual insert action
        INSERT INTO EMPLOYEES (EMPLOYEE_ID, LAST_NAME, SALARY)
        VALUES (:NEW.EMPLOYEE_ID, :NEW.LAST_NAME, :NEW.SALARY);

    ELSIF UPDATING THEN
        sal_diff := :NEW.SALARY - :OLD.SALARY;
        DBMS_OUTPUT.PUT_LINE('Updating Employee: ' || :NEW.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Old salary = ' || :OLD.SALARY || ', New salary = ' || :NEW.SALARY);
        DBMS_OUTPUT.PUT_LINE('Difference: ' || sal_diff);
        -- Perform actual update action
        UPDATE EMPLOYEES
        SET SALARY = :NEW.SALARY
        WHERE EMPLOYEE_ID = :OLD.EMPLOYEE_ID;

    ELSIF DELETING THEN
        DBMS_OUTPUT.PUT_LINE('Deleting Employee: ' || :OLD.LAST_NAME);
        DBMS_OUTPUT.PUT_LINE('Final Salary: ' || :OLD.SALARY);
        -- Perform actual delete action
        DELETE FROM EMPLOYEES WHERE EMPLOYEE_ID = :OLD.EMPLOYEE_ID;
    END IF;
END;
/
```

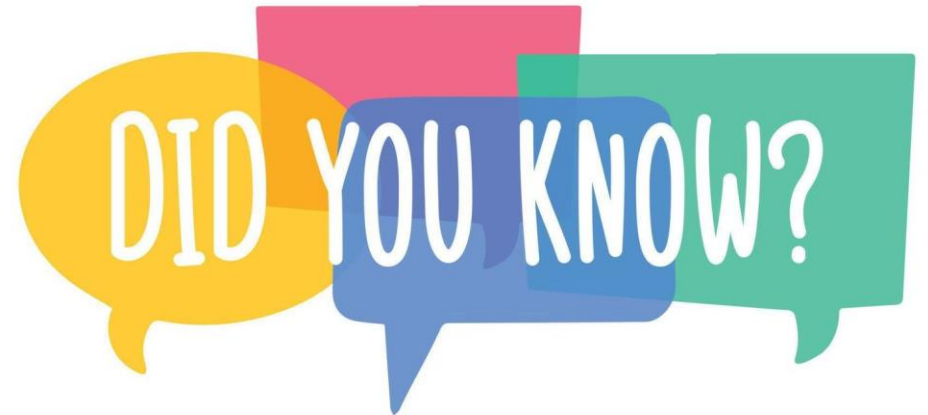
Instead Of Event: Log Employee Salary Changes and Operations – Solution

Explanation of Changes:

1. **View:** The trigger operates on EMPLOYEES_VIEW, not the EMPLOYEES table.
2. **INSTEAD OF Trigger:** The INSTEAD OF trigger intercepts the INSERT, UPDATE, and DELETE operations on the view and performs custom actions (like logging the changes and updating the EMPLOYEES table) instead of directly executing them.
3. **DBMS_OUTPUT:** It logs the salary change information for debugging or informational purposes.
4. **Custom Actions:** You can add custom actions for logging or other purposes, and after logging, the actual INSERT, UPDATE, or DELETE is performed on the EMPLOYEES table.

Important Considerations:

- The INSTEAD OF trigger replaces the default behavior for the view, meaning it doesn't directly modify the view's data. Instead, it performs the actual operations on the underlying EMPLOYEES table.
- You must use INSTEAD OF triggers only on views. They cannot be applied directly to tables.



Row-Level Trigger

To illustrate a row-level trigger using the **EMPLOYEES** table, let's create a trigger that logs salary changes for each employee affected by an UPDATE statement. This example will include a condition that only logs the change if the employee's role is not "President" (assuming the President's role has a ROLE_ID of 1).

Trigger Example: Log Salary Changes on Update

Here's the row-level trigger that logs salary changes each time an UPDATE operation affects the **EMPLOYEES** table.

```
CREATE OR REPLACE TRIGGER log_salary_changes
  BEFORE UPDATE OF SALARY ON EMPLOYEES -- Trigger fires before each salary update
  FOR EACH ROW -- Row-level trigger to fire once per affected row
  WHEN (NEW.ROLE_ID <> 1) -- Condition to skip the Manager's role, assumed to be ROLE_ID = 1
  DECLARE
    sal_diff NUMBER; -- Variable to hold the difference between old and new salary
  BEGIN
    -- Calculate the salary difference
    sal_diff := :NEW.SALARY - :OLD.SALARY;

    -- Output employee name and salary details
    DBMS_OUTPUT.PUT_LINE('Employee: ' || :NEW.LAST_NAME);
    DBMS_OUTPUT.PUT_LINE('Old salary = ' || :OLD.SALARY || ', New salary = ' || :NEW.SALARY);
    DBMS_OUTPUT.PUT_LINE('Difference: ' || sal_diff);
  END;
/
```

Explanation of the Trigger

Trigger Events:

The trigger fires **before** each UPDATE operation on the **SALARY** column of the **EMPLOYEES** table.

Row-Level Trigger:

The FOR EACH ROW clause makes it a row-level trigger, meaning it will execute once for each row that is updated.

WHEN Condition:

The WHEN (NEW.ROLE_ID <> 1) condition ensures that this trigger does not fire for employees with a ROLE_ID of 1 (assumed to be the Manager).

Logging Salary Change:

The trigger calculates the difference between the new and old salary (sal_diff) and uses DBMS_OUTPUT.PUT_LINE to output the employee's last name and salary details.

Execution Order of Triggers

In Oracle, it is possible to define multiple triggers for the same timing point (e.g., BEFORE INSERT, AFTER UPDATE). However, **prior to Oracle 11g**, there was no guaranteed execution order for such triggers. This meant that Oracle could execute them in any order, which could lead to inconsistent behavior in complex systems. Starting with **Oracle 11g**, the **FOLLOWS** clause was introduced. This clause allows developers to explicitly specify the execution order of triggers defined for the same timing point. By using the FOLLOWS clause, you can ensure predictable and consistent trigger behavior.

Example: Using the FOLLOWS Clause

The following example demonstrates how to create a table and two triggers for the same timing point (BEFORE INSERT). The FOLLOWS clause is used to guarantee their execution order.

```
-- Create a table to test trigger execution
CREATE TABLE trigger_follows_test (
  id      NUMBER,      -- Unique identifier for each row
  description VARCHAR2(50) -- Description for each row
);
```

Step 1: Create a Table

The table trigger_follows_test will be used to test the triggers.

```
-- Create the first trigger
CREATE OR REPLACE TRIGGER trigger_follows_test_trg_1
BEFORE INSERT ON trigger_follows_test -- Trigger fires before an insert
FOR EACH ROW                          -- Fires for each row inserted
BEGIN
  -- Print message indicating this trigger was executed
  DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_1 - Executed');
END;
/

-- Create the second trigger
CREATE OR REPLACE TRIGGER trigger_follows_test_trg_2
BEFORE INSERT ON trigger_follows_test -- Trigger fires before an insert
FOR EACH ROW                          -- Fires for each row inserted
BEGIN
  -- Print message indicating this trigger was executed
  DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_2 - Executed');
END;
/
```

Step 2: Create Two Triggers Without Execution Order

Initially, two triggers are created for the same timing point (BEFORE INSERT), but the execution order is not defined.

Execution Order of Triggers – Test Trigger

```
-- Enable output to see DBMS_OUTPUT messages
SQL> SET SERVEROUTPUT ON

-- Insert a row into the table
INSERT INTO trigger_follows_test VALUES (1, 'ONE');

-- Output: The execution order is not guaranteed
-- Example:
-- TRIGGER_FOLLOWS_TEST_TRG_2 - Executed
-- TRIGGER_FOLLOWS_TEST_TRG_1 - Executed
-- (or the reverse order)
```

Step 3: Test the Triggers Without Defined Order

Perform an INSERT operation to observe the execution order of the triggers.

```
-- Recreate the first trigger with the FOLLOWS clause
CREATE OR REPLACE TRIGGER trigger_follows_test_trg_1
BEFORE INSERT ON trigger_follows_test    -- Trigger fires before an insert
FOR EACH ROW                            -- Fires for each row inserted
FOLLOWS trigger_follows_test_trg_2      -- Ensures this trigger runs after the
BEGIN
    -- Print message indicating this trigger was executed
    DBMS_OUTPUT.put_line('TRIGGER_FOLLOWS_TEST_TRG_1 - Executed');
END;
/
```

Step 4: Use the FOLLOWS Clause to Specify Execution Order

Recreate TRIGGER_FOLLOWS_TEST_TRG_1 with the FOLLOWS clause to ensure it always executes **after** TRIGGER_FOLLOWS_TEST_TRG_2.

```
-- Insert another row into the table
INSERT INTO trigger_follows_test VALUES (2, 'TWO');

-- Output: The execution order is now guaranteed
-- TRIGGER_FOLLOWS_TEST_TRG_2 - Executed
-- TRIGGER_FOLLOWS_TEST_TRG_1 - Executed
```

Step 5: Test the Triggers with Defined Order

Perform another INSERT operation to confirm the execution order is now guaranteed.

```
-- Drop the test table to remove all triggers and data
DROP TABLE trigger_follows_test;
```

Step 6: Clean Up

Drop the table to clean up the environment.

Summary:

Without the FOLLOWS Clause:

The execution order of triggers is unpredictable.

With the FOLLOWS Clause:

You can explicitly define the execution order of triggers for the same timing point, ensuring predictable behavior.

The FOLLOWS clause is especially useful in complex systems where multiple triggers depend on the output of one another.

Why Use Compound Triggers? (1/2)

A **compound trigger** in Oracle PL/SQL is a special type of trigger that allows you to **define multiple timing points within a single trigger**. It is particularly **useful when you need to handle multiple operations** (e.g., INSERT, UPDATE, DELETE) and when operations span different timing points (e.g., BEFORE or AFTER) for the same triggering event.

Share Data Across Timing Points

- Compound triggers allow you to share data between different timing points (BEFORE STATEMENT, BEFORE EACH ROW, AFTER EACH ROW, and AFTER STATEMENT) within a single trigger, improving consistency and making complex actions simpler.

Improve Performance with Bulk Inserts

- Instead of inserting data row-by-row, compound triggers can collect data during row-level actions and perform bulk inserts later in the AFTER STATEMENT section, enhancing performance.

Avoid Mutating Table Errors

- By using temporary data storage, compound triggers help avoid mutating table errors, which occur when a trigger tries to modify or query the table that caused it to fire.

Why Use Compound Triggers? (2/2)

Restrictions of Compound Triggers

- **Must Be a DML Trigger:** Only INSERT, UPDATE, or DELETE operations on tables or views can have compound triggers.
- **Trigger Body Must Be a Compound Trigger Block:** The body must use the compound trigger structure, with sections for different timing points.
- **No Initialization Block:** You cannot have an initialization block. All data structures must be initialized in the declaration or timing sections.
- **OLD and NEW Cannot Be Used Outside Row-Level Sections:** :OLD and :NEW can only be accessed in the BEFORE EACH ROW and AFTER EACH ROW sections, not in DECLARE, BEFORE STATEMENT, or AFTER STATEMENT.
- **Firing Order Is Not Guaranteed:** If multiple compound triggers exist on the same table, their firing order is not guaranteed unless explicitly defined using FOLLOWS or PRECEDES.

Best Practices for Writing Compound Triggers

- **Use Shared Variables:** Declare shared variables in the trigger body to hold data across timing points.
- **Leverage Bulk Processing:** Collect data in row-level actions and process it in bulk during AFTER STATEMENT for better performance.
- **Keep Logic Simple and Modular:** Avoid complex logic in any one timing point and break up actions logically across sections.
- **Handle Mutating Table Errors:** Store data temporarily and perform operations after the statement to avoid mutating table issues.

Advantages of Compound Triggers

- **Simplified Maintenance:** Consolidates multiple timing points in one trigger, reducing the need for separate triggers and simplifying maintenance.
- **Performance Boost:** Bulk operations improve performance by reducing row-by-row processing.
- **Error Prevention:** Helps prevent common issues like mutating table errors.
- **Flexibility:** Multiple timing points offer flexibility to execute actions at different stages of the DML operation.

Syntax

```
CREATE OR REPLACE TRIGGER trigger_name  
FOR [INSERT | UPDATE | DELETE] ON table_name  
COMPOUND TRIGGER
```

```
-- Declare shared variables here  
DECLARE  
    variable_name datatype;
```

```
-- Before Statement Block  
BEFORE STATEMENT IS  
BEGIN  
    -- Code executed once before the triggering statement  
END BEFORE STATEMENT;
```

```
-- Before Each Row Block  
BEFORE EACH ROW IS  
BEGIN  
    -- Code executed before each row operation  
END BEFORE EACH ROW;
```

```
-- After Each Row Block  
AFTER EACH ROW IS  
BEGIN  
    -- Code executed after each row operation  
END AFTER EACH ROW;
```

```
-- After Statement Block  
AFTER STATEMENT IS  
BEGIN  
    -- Code executed once after the triggering statement  
END AFTER STATEMENT;
```

Optional Section

Syntax Explanation:

- This syntax shows the creation of '**COMPOUND**' trigger.
- Declarative section is common for all the execution block in the trigger body.
- These 4 timing blocks can be in any sequence. It is not mandatory to have all these 4 timing blocks. We can create a COMPOUND trigger only for the timings which are required.

Syntax:

```
CREATE [ OR REPLACE ] TRIGGER <trigger_name>  
FOR  
[ INSERT | UPDATE | DELETE.....]  
ON <name of underlying object>  
<Declarative part>
```

```
BEFORE STATEMENT IS  
BEGIN  
    <Execution part>;  
END BEFORE STATEMENT;
```

BEFORE-
STATEMENT Level

```
BEFORE EACH ROW IS  
BEGIN  
    <Execution part>;  
END EACH ROW;
```

BEFORE-
ROW Level

```
AFTER EACH ROW IS  
BEGIN  
    <Execution part>;  
END AFTER EACH ROW;
```

AFTER-ROW
Level

```
AFTER STATEMENT IS  
BEGIN  
    <Execution part>;  
END AFTER STATEMENT;
```

AFTER-
STATEMENT Level

```
END;
```

Source Image: <https://www.guru99.com/triggers-pl-sql.html>

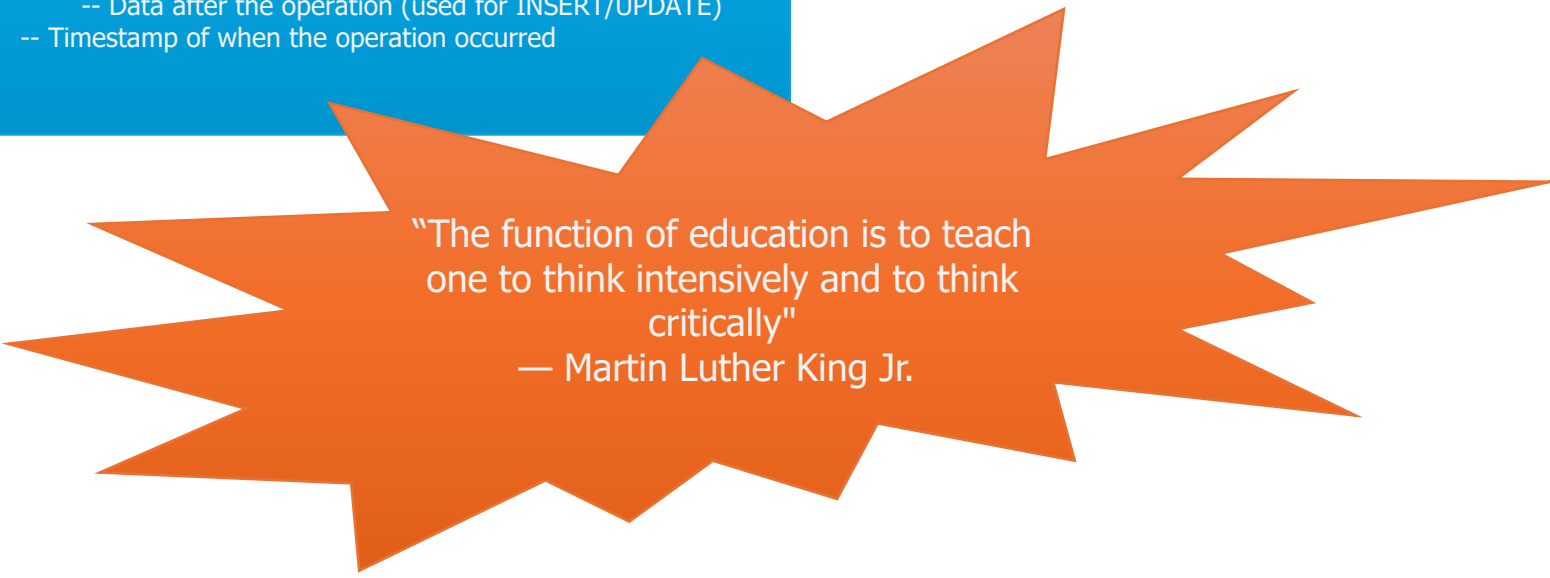
```
END trigger_name;
```

Example: Auditing Data Changes with a Compound Trigger

Suppose we have a table EMPLOYEES and want to log changes (inserts, updates, and deletes) to an audit table AUDIT_EMP.



```
-- Table to store audit logs for employee operations
CREATE TABLE audit_emp (
  audit_id NUMBER GENERATED BY DEFAULT AS IDENTITY, -- Unique identifier for each audit record
  operation_type VARCHAR2(20), -- Type of DML operation (INSERT, UPDATE, DELETE)
  employee_id NUMBER, -- ID of the affected employee
  old_data VARCHAR2(1000), -- Data before the operation (used for UPDATE/DELETE)
  new_data VARCHAR2(1000), -- Data after the operation (used for INSERT/UPDATE)
  timestamp DATE -- Timestamp of when the operation occurred
);
```



"The function of education is to teach
one to think intensively and to think
critically"
— Martin Luther King Jr.

Audit Salary Updates with Shared Object Type (1/2)

Scenario: Audit Salary Updates with Shared Object Type

We'll track salary updates in an **audit table** using a compound trigger. An **object type** will hold information about employees whose salaries are updated.

```
/*
The audit table
*/

CREATE TABLE salary_audit (
  employee_id NUMBER,
  old_salary NUMBER,
  new_salary NUMBER,
  change_date DATE
);

/*
Create an Object Type
*/
CREATE OR REPLACE TYPE employee_salary_obj AS OBJECT (
  employee_id NUMBER,
  old_salary NUMBER,
  new_salary NUMBER
);

/*
Create a Nested Table to Store Objects
*/

CREATE OR REPLACE TYPE employee_salary_tab IS TABLE OF employee_salary_obj;
```



Audit Salary Updates with Shared Object Type (2/2)

```
CREATE OR REPLACE TRIGGER compound_salary_audit
FOR UPDATE OF SALARY ON EMPLOYEES
COMPOUND TRIGGER
```

```
-- Variable declaration: Nested table to store salary changes during the transaction
salary_changes employee_salary_tab := employee_salary_tab();
```

```
-- BEFORE EACH ROW section: Capture the OLD and NEW salary values
BEFORE EACH ROW IS
BEGIN
```

```
-- Collect the old and new salary details into the nested table
salary_changes.EXTEND; -- Extend the nested table
salary_changes(salary_changes.COUNT) := employee_salary_obj(
    :OLD.EMPLOYEE_ID,
    :OLD.SALARY,
    :NEW.SALARY
);
END BEFORE EACH ROW;
```

```
-- AFTER STATEMENT section: Log all salary changes into the audit table
AFTER STATEMENT IS
BEGIN
```

```
    FOR i IN 1 .. salary_changes.COUNT LOOP
        INSERT INTO salary_audit (
            employee_id,
            old_salary,
            new_salary,
            change_date
        ) VALUES (
            salary_changes(i).employee_id,
            salary_changes(i).old_salary,
            salary_changes(i).new_salary,
            SYSDATE
        );
    END LOOP;
```

```
-- Clear the nested table after processing
salary_changes.DELETE;
END AFTER STATEMENT;
```

```
END compound_salary_audit;
/
```

```
/*
Test the trigger
*/
```

```
-- Update salaries for multiple employees
UPDATE EMPLOYEES
SET SALARY = SALARY * 1.10
WHERE DEPARTMENT_ID = 3;
```

```
-- Check the audit table for the logged changes
SELECT * FROM salary_audit;
```

Problem Statement Example – Compound of Maintaining Employee Salary Records Automatically

Scenario:

Your company maintains two tables:

1. **EMPLOYEES**: Stores EMPLOYEE_ID, SALARY, and BONUS.
2. **EMPLOYEE_SALARIES**: Tracks employee salary and bonus details for payroll.

Requirement:

Whenever an employee is added or their SALARY or BONUS is updated in the EMPLOYEES table, ensure that the EMPLOYEE_SALARIES table is updated to reflect the changes. Use a **compound trigger** to manage this logic efficiently. If the employee does not exist in EMPLOYEE_SALARIES, insert a new record.

Database Schema:

EMPLOYEES:

- EMPLOYEE_ID (Primary Key)
- SALARY
- BONUS

EMPLOYEE_SALARIES:

- EMPLOYEE_ID (Primary Key)
- SALARY
- BONUS



Tasks:

1. Write a Compound Trigger:

- Use a compound trigger to ensure the following:
 - During the **statement-level operation**, initialize necessary structures (if needed).
 - At the **row-level operation**, update or insert into the EMPLOYEE_SALARIES table as required.
 - Log any errors explicitly.
- Handle duplicate records gracefully to avoid conflicts in the EMPLOYEE_SALARIES table.

2. Add Exception Handling:

- Explicitly handle exceptions for UPDATE or INSERT operations.
- Log meaningful error messages for failures (e.g., primary key violations).

3. Test the Trigger:

- Insert a new employee into the EMPLOYEES table and verify changes in EMPLOYEE_SALARIES.
- Update an existing employee's SALARY or BONUS in EMPLOYEES and ensure the changes are reflected in EMPLOYEE_SALARIES.

Validation Steps:

- Insert a new employee into EMPLOYEES and check the insertion into EMPLOYEE_SALARIES.
- Update an employee's salary or bonus and verify the update in EMPLOYEE_SALARIES.
- Test duplicate handling by attempting to insert an already existing record in EMPLOYEE_SALARIES.

Auditing Student Data with a Compound Trigger at AUCA

Problem Statement:

You are a database administrator for ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA) system. Your task is to track changes made to student data in a **Students Audit Table** using a **compound trigger**. This ensures that any updates to student records are logged for auditing purposes. Follow the steps below to design and implement the system:

1. Create two tables:

- A students table for storing student records.
- A students_audit table for logging updates to the students table.

2. Insert sample data into the students table for testing purposes.

▪ Define a compound trigger:

- Use this trigger to log the student_id, the type of action performed (e.g., UPDATE), and the date of the change into the students_audit table.
- Ensure efficient bulk processing using collections.

1. Test your implementation:

- Update records in the students table and verify that the changes are logged in the students_audit table.



Auditing Student Data with a Compound Trigger at AUCA – Solution (1/3)

```
/*
1. Create Student Table
*/

CREATE TABLE students (
    student_id NUMBER PRIMARY KEY,
    student_name VARCHAR2(100),
    major VARCHAR2(50),
    gpa NUMBER(3, 2)
);

/*
2. Student audit table
*/
CREATE TABLE students_audit (
    audit_id NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    student_id NUMBER,
    action VARCHAR2(50),
    change_date DATE
);

/*
Step 2: Insert Sample Data
Populate the students table with sample data:
*/
BEGIN
    FOR i IN 1..1000 LOOP
        INSERT INTO students (student_id, student_name, major, gpa)
        VALUES (
            i,
            'Student ' || i,
            CASE MOD(i, 4)
                WHEN 0 THEN 'Computer Science'
                WHEN 1 THEN 'Engineering'
                WHEN 2 THEN 'Business'
                ELSE 'Arts'
            END,
            2.5 + MOD(i, 5) * 0.1
        );
    END LOOP;
END;

SELECT * FROM students_audit;
```

Auditing Student Data with a Compound Trigger at AUCA – Solution (2/3)

```
-- Step 4: Create a compound trigger to log updates on the students table
CREATE OR REPLACE TRIGGER student_update_compound_trigger
FOR UPDATE ON students
COMPOUND TRIGGER

    -- Declare a collection for bulk processing
    TYPE audit_data_type IS TABLE OF students_audit%ROWTYPE;
    audit_data audit_data_type := audit_data_type();

    BEFORE STATEMENT IS
    BEGIN
        DBMS_OUTPUT.PUT_LINE('Student update process started.');
```

END BEFORE STATEMENT;

```
    AFTER EACH ROW IS
    BEGIN
        -- Add row data to the collection for bulk processing
        audit_data.EXTEND;
        audit_data(audit_data.LAST).audit_id := AUDIT_SEQ.NEXTVAL; -- Use sequence for audit_id
        audit_data(audit_data.LAST).student_id := :OLD.student_id; -- Log the student_id
        audit_data(audit_data.LAST).action := 'UPDATE';           -- Action type
        audit_data(audit_data.LAST).change_date := SYSDATE;      -- Timestamp of the change
    END AFTER EACH ROW;
```

```
    AFTER STATEMENT IS
    BEGIN
        -- Perform bulk insert into the audit table
        IF audit_data.COUNT > 0 THEN
            FORALL i IN 1..audit_data.COUNT
                INSERT INTO students_audit (
                    audit_id, student_id, action, change_date
                ) VALUES (
                    audit_data(i).audit_id,
                    audit_data(i).student_id,
                    audit_data(i).action,
                    audit_data(i).change_date
                );
            END IF;
        DBMS_OUTPUT.PUT_LINE('Student update process completed.');
```

END AFTER STATEMENT;

```
END student_update_compound_trigger;
/
```

Auditing Student Data with a Compound Trigger at AUCA – Solution (3/3) – Test the trigger

```
/*
```

Step 5: Test the Trigger

Now, we need to test the trigger by performing an update on some student records and verifying that the updates are logged into the `students_audit` table.

```
*/
```

```
-- Step 5: Test the trigger by updating some student records
```

```
UPDATE students SET gpa = gpa + 0.5 WHERE gpa < 4.0;
```

```
-- Verify the number of audit records logged
```

```
SELECT COUNT(*) FROM students_audit;
```

```
-- View the first 10 records in the students_audit table to check the logged actions
```

```
SELECT * FROM students_audit WHERE ROWNUM <= 10;
```

Scenario: Managing Allowances Based on Attendance

Here's a scenario that incorporates the creation and use of a compound trigger to manage related data across the ALLOWANCES and ATTENDANCE tables. The trigger ensures that when an employee's attendance record indicates absence, an allowance record is automatically marked as not applicable (IS_APPLICABLE = 'N').

Scenario: Managing Allowances Based on Attendance

Objective:

Ensure that allowances are correctly updated based on employee attendance records. If an employee is marked absent on a particular date, all allowance entries linked to their role and effective on or before that date are marked as not applicable (IS_APPLICABLE = 'N').

Steps

1. Tables Involved:

- ALLOWANCES: Tracks allowances assigned to roles.
- ATTENDANCE: Records daily attendance for employees.

2. Business Rule:

- If an employee is marked as absent (STATUS = 'Absent') on a specific date:
 - All allowances tied to the employee's role (ROLE_ID) and effective on or before the absence date (EFFECTIVE_DATE) must have their IS_APPLICABLE field updated to 'N'.

3. Technical Requirements:

- Use a **compound trigger** to handle the update in a transaction-safe manner.
- Use an **object type** to manage allowances updates as part of the trigger logic.



Recap: PLSQL - Trigger

1. DML Triggers

- These triggers are executed in response to **Data Manipulation Language (DML)** operations:
- **Insert**
- **Update**
- **Delete**

4. DDL Triggers

- These triggers respond to **Data Definition Language (DDL)** events, such as:
- **Drop**
- **Alter**
- **Truncate**
- **Grant**
- **Revoke**
- **Rename**

5. System Triggers

- These are triggered based on system-level events like:
- **Logon**
- **Logoff**
- **Startup**
- **Shutdown**

2. Instead Of Trigger

- Designed for **views** to handle operations (like insert, update, delete) that are otherwise not directly allowed on views.

3. Compound Trigger

- Combines all **DML triggers** (insert, update, delete) under a single trigger.
- Helps manage complex logic by consolidating related operations.

Problem Statement – Create a Trigger

Question:

You need to create a trigger in an Oracle database that automatically increases the salary of employees who were hired in 2022 by 22.5% of their base salary. This trigger should activate after an INSERT or UPDATE operation on the employees table to ensure that any new employee hired in 2022 receives the salary increase right away.



Solution: Create Trigger trg_increase_salary

Solution

To accomplish this, you will define a **DML Trigger** that will execute after a new employee is inserted or an existing employee's hiring date is updated. The trigger will check if the hiring date falls within the year 2022 and, if so, apply the 22.5% increase to the employee's salary.

Here's how you can implement this in PL/SQL:

1. Assumptions:

- The employees table has at least the following columns: employee_id, hire_date, and salary.
- The salary column stores the base salary of the employee.

2. Trigger Creation:

```
-- Trigger to increase the salary of employees hired in 2022 by 22.5%
CREATE OR REPLACE TRIGGER trg_increase_salary
BEFORE INSERT OR UPDATE ON employees
FOR EACH ROW
BEGIN
    BEGIN
        -- Check if the employee was hired in 2022
        IF EXTRACT(YEAR FROM :NEW.hire_date) = 2022 THEN
            -- Increase salary by 22.5%
            :NEW.salary := :NEW.salary * 1.225;
        END IF;
    EXCEPTION
        WHEN OTHERS THEN
            -- Log the error or handle it as needed
            DBMS_OUTPUT.PUT_LINE('Error in trg_increase_salary: ' ||
SQLERRM);
            -- Optionally, raise the exception if you want to stop the
operation
            RAISE;
        END;
    END;
/
```



Explanation:

1. Nested BEGIN...EXCEPTION...END Block: The exception handling is wrapped around the core logic. If any error occurs (e.g., due to data type issues), it will be caught.

2. DBMS_OUTPUT.PUT_LINE: This outputs the error message to the console, making debugging easier.

3. RAISE (optional): The RAISE statement re-throws the exception if you want the error to halt the transaction. You can remove it if you just want to log the error without stopping the operation.

Testing the Trigger - trg_increase_salary

Testing the Trigger

You can test the trigger by inserting a new employee or updating an existing employee's hire date. Here's an example of how to do this:

```
-- Insert a new employee hired in 2022
INSERT INTO employees (employee_id, hire_date, salary)
VALUES (1, TO_DATE('2022-06-15', 'YYYY-MM-DD'), 50000);

-- Check the salary after the trigger fires
SELECT * FROM employees WHERE employee_id = 1;

-- Update an existing employee's hire date to 2022
UPDATE employees
SET hire_date = TO_DATE('2022-07-20', 'YYYY-MM-DD')
WHERE employee_id = 1;

-- Check the salary after the update
SELECT * FROM employees WHERE employee_id = 1;
```

Exercise: Enhancing Accountability and Insights Through Employee Salary Change Tracking

Exercise - Problem statement that relates to the trigger functionality and its implications in a business context:

Problem Statement

- **Context:** In a dynamic business environment, managing employee compensation is crucial for maintaining employee satisfaction and retention. Changes to employee salaries occur frequently due to performance reviews, promotions, or market adjustments. Therefore, it is vital for the HR department to have a transparent mechanism for tracking and reviewing these salary changes.
- **Problem:** How can the organization effectively monitor and analyze salary adjustments made to employees in the **EMPLOYEES** table to ensure accountability and informed decision-making?

Questions to Consider:

1. How can we log each salary change with relevant details (old salary, new salary, and salary difference) to maintain a historical record of salary adjustments?
2. What implications do salary changes have on overall employee performance and morale within the organization?
3. How can the HR team leverage the logged salary changes to identify trends in compensation, assess equity among employees, and make strategic adjustments to compensation structures?
4. In what ways can we ensure that the logging process does not introduce performance overhead, especially during bulk updates?

Objective

The objective of this inquiry is to develop a robust mechanism (such as the **display_salary_changes** trigger) to automatically log salary changes in the **EMPLOYEES** table. This will provide the HR team with real-time insights into salary adjustments, enabling them to address compensation-related issues proactively and foster a fair workplace environment.

Solution: Employee Salary Change Tracking

--Step 1: Create the trigger to log salary changes

```
CREATE OR REPLACE TRIGGER display_salary_changes
-- Trigger fires before DELETE, INSERT, or UPDATE operations on the
EMPLOYEES table
BEFORE DELETE OR INSERT OR UPDATE ON EMPLOYEES
-- This is a row-level trigger
FOR EACH ROW
-- Condition to check if the new employee ID is valid
WHEN (NEW.EMPLOYEE_ID > 0)
DECLARE
    sal_diff NUMBER; -- Variable to hold the salary difference
BEGIN
    -- Check if the operation is an update
    IF UPDATING THEN
        -- Calculate the difference between new and old salary
        sal_diff := :NEW.SALARY - :OLD.SALARY;
        -- Output old salary to the console
        dbms_output.put_line('Old salary: ' || :OLD.SALARY);
        -- Output new salary to the console
        dbms_output.put_line('New salary: ' || :NEW.SALARY);
        -- Output the salary difference to the console
        dbms_output.put_line('Salary difference: ' || sal_diff);
    END IF;
END;
/
```

Test: Employee Salary Change Tracking

-- Step 2: Insert sample employee data for testing

```
INSERT INTO EMPLOYEES (EMPLOYEE_ID, FIRST_NAME, LAST_NAME, EMAIL, PHONE_NUMBER, ADDRESS, HIRE_DATE, DEPARTMENT_ID, ROLE_ID, SALARY, BONUS)
VALUES (1, 'John', 'Doe', 'john.doe@example.com', '1234567890', '123 Main St', SYSDATE, 1, 1, 50000, 5000);
```

-- Step 3: Update the salary to trigger the logging mechanism

```
UPDATE EMPLOYEES
SET SALARY = 60000 -- New salary amount
WHERE EMPLOYEE_ID = 1; -- Update for the employee with ID 1
```

-- Step 4: Verify that the trigger has logged the information

```
-- Enable output display to see the logs from dbms_output
SET SERVEROUTPUT ON;
```

-- This command will display the logs output generated by the trigger in the console

-- The logs will show the old salary, new salary, and the salary difference for employee with ID 1

-- Step 5: Query to analyze salary changes logged in a hypothetical Emp_log table

-- (Assuming an Emp_log table exists to keep track of salary changes)

```
SELECT
    Emp_id,           -- Employee ID
    Log_date,         -- Date of the log entry
    New_salary,       -- New salary amount
    Role_ID,          -- Employee role ID
    Comments,         -- Any comments associated with the change
    Action            -- Action taken (e.g., 'New Salary')
FROM
    Emp_log           -- Log table that stores changes
WHERE
    Emp_id = 1;       -- Filter for the specific employee
```

-- This query retrieves logged changes for the employee with ID 1

-- It can be used by HR to review salary changes and ensure transparency in the compensation process

Question

Question:

In a corporate environment, it's essential to monitor salary changes for employees to ensure fair compensation practices and maintain transparency. A trigger can be used to log salary change information for all employees except for high-ranking officials, such as the President.

Using your EMPLOYEES table, create a conditional trigger named `log_salary_changes` that fires before any DELETE, INSERT, or UPDATE operation. The trigger should only log salary changes for employees who do not hold the position of 'President'. It should output the last name of the employee, their old salary, new salary, and the difference in salary.

The trigger should have the following specifications:

- **When to Fire:** The trigger should only activate if the employee's job ID is not 'AD_PRES'.
- **Output Format:** The output should state the employee's last name, old salary, new salary, and the calculated difference.

Example Trigger Implementation:



Difference Between Cursor and Trigger in PL/SQL

1. Cursor in PL/SQL

- **Definition:** Points to a memory area holding query results.
 - **Types:**
 - **Implicit Cursor:** Automatically created for simple queries.
 - **Explicit Cursor:** User-defined for complex queries.
 - **Advantages:** Enables row-by-row validation and transaction control.
 - **Disadvantages:** Uses extra resources, which can increase network load.
-
- **2. Trigger in PL/SQL Definition:** A program that executes on events like INSERT, UPDATE, and DELETE.
 - **Event Types:** BEFORE and AFTER events.
 - **Advantages:** Automates monitoring, helps enforce data integrity.
 - **Disadvantages:** Can complicate debugging and may impact performance if overused.

3. Trigger in PL/SQL

Cursor	Trigger
Processes rows one by one.	Executes automatically on events.
Can be defined inside a trigger.	Cannot be created within a cursor.
Created during SQL execution.	Stored in the database, triggered by events.
Activated by SQL statements.	Triggered by DDL/DML events.

4. Conclusion

- **Cursors:** Best for controlled, row-by-row data processing.
- **Triggers:** Ideal for automating responses to database events to maintain data consistency.



Application of Triggers

Applications of Triggers

Triggers are used to automatically execute a specified PL/SQL code block in response to certain events on a database table or view, such as INSERT, UPDATE, DELETE, or DDL operations like CREATE, ALTER, or DROP. Triggers ensure data consistency, enforce business rules, audit changes, and manage complex data transformations without requiring explicit instructions from the user.

Common Applications of Triggers

Auditing Data Changes: Automatically log any changes made to important tables. For example, each time an employee's salary is updated, a trigger could log the old and new salaries, the employee ID, and the timestamp in an audit table.

Enforcing Business Rules: Ensure compliance with business rules. For instance, a trigger might prevent inserting an order if the customer's credit limit is exceeded.

Maintaining Derived Values: Automatically update derived or calculated fields. For example, if an employee's salary is updated, a trigger could recalculate the BONUS or TAX fields.

Complex Data Validation: Implement validations that go beyond what standard constraints can enforce. For example, preventing deletion of a parent record if child records exist, which would require a cascading delete.

Synchronizing Data: Maintain consistency across related tables. For instance, a trigger could ensure that when an employee is deleted from an EMPLOYEES table, corresponding records in related tables are also removed.

Keep Learning!